



ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ

Ακαδημαϊκό Έτος 2022-2023

ΤΕΧΝΙΚΗ ΑΝΑΦΟΡΑ

1^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ

ΚΥΡΙΑΚΑΤΟΥ ΝΙΚΗ ΑΙΚΑΤΕΡΙΝΗ

A.M.: 1084624

ΚΥΡΙΑΚΑΤΟΥ ΜΑΡΙΑ ΑΝΑΣΤΑΣΙΑ

A.M.: 1059656

ΖΥΓΟΥΡΗΣ ΦΙΛΙΠΠΟΣ ΠΑΡΑΣΚΕΥΑΣ

A.M.: 1084660

Πάτρα 2022-23

1^η ΆΣΚΗΣΗ

Το `#!/bin/bash` αποτελεί την εντολή shebang και χρησιμοποιείται για να καθορίσει τον interpreter που θα χρησιμοποιηθεί με σκοπό να εκτελέσει το script.

```
#!/bin/bash
```

Ερώτημα i

Δηλώνουμε στην αρχή του κώδικα σε σχόλια τα ονοματεπώνυμα των μελών της ομάδας και για το κάθε μέλος ξεχωριστά αναθέτουμε το αντίστοιχο AM του σε μια μεταβλητή `stdiAM`.

```
#Team Members
```

```
#Filippos-Paraskevas Zygouris
```

```
std1AM="1084660"
```

```
#Niki-Aikaterinh Kyriakatou
```

```
std2AM="1084624"
```

```
#Maria-Anastasia Kyriakatou
```

```
std3AM="1059656"
```

Αρχικά:

- ✓ Ελέγχουμε εάν ένα αρχείο έχει παρασχεθεί ως όρισμα στη γραμμή εντολών κατά την διαδικασία εκτέλεσης του σεναρίου.
- ✓ Με την `if` ελέγχουμε εάν ο αριθμός των ορισμάτων που διαβιβάζονται στο σενάριο (`$#`) ισούται με 0 και το `-eq` είναι ένας τελεστής σύγκρισης που επιστρέφει `true` με την προϋπόθεση ότι οι δύο τελεστές είναι ίσοι.

Εάν δεν έχει δοθεί κάποιο αρχείο ως όρισμα, τότε:

- ✓ Εκτελείται ο κώδικας μέσα στο `if statement` όπου εμφανίζονται τα αναγνωριστικά των μελών της ομάδας μέσω της εντολής `echo "$std1AM|$std2AM|$std3AM"`.
- ✓ Εκτελείται η εντολή `exit 0` η οποία διακόπτει την εκτέλεση του script.

Εάν δόθηκε κάποιο αρχείο ως όρισμα, τότε :

- ✓ Η εκτέλεση του script συνεχίζεται στις επόμενες γραμμές κώδικα.

```
#Check if a file was provided as an argument in the command line
if [ $# -eq 0 ];
then
#No file provided ,so display team members IDs
echo "$std1AM|$std2AM|$std3AM"
exit 0
fi
```

Έπειτα, για καθ' ένα από τα παρακάτω ερωτήματα ελέγχουμε εάν το αρχείο που δόθηκε από τον χρήστη έχει επέκταση ονόματος .log . Εάν δεν είναι αυτής της μορφής θα εμφανίζει τα μηνύματα «Wrong File Argument» και «Instead use the correct form \$filename.log» . Έτσι του επιδεικνύουμε ότι έχει κάνει λάθος με αποτέλεσμα να χρησιμοποιήσει την σωστή επέκταση (.log) στο όνομα του αρχείου που έχει ήδη δώσει.

Επιπλέον, Η εντολή if ["\${filename: -4}" != ".log"]; ελέγχει αν οι τελευταίοι 4 χαρακτήρες του ονόματος του αρχείου (η επέκταση του) δεν είναι ".log". Το \${filename: -4} επιστρέφει τους τελευταίους 4 χαρακτήρες της μεταβλητής filename.

Εάν η σύγκριση είναι αληθής (το αρχείο δεν έχει επέκταση ".log"), τότε:

- ✓ εκτελείται ο κώδικας μέσα στο if όπως ήδη αναφέρθηκε παραπάνω και εκτελείται η εντολή exit 1 η οποία διακόπτει την εκτέλεση του script.

Εάν όμως η επέκταση του αρχείου είναι σωστή:

- ✓ η εκτέλεση του script συνεχίζεται κανονικά στις επόμενες γραμμές του κώδικα.

```
#Check if the log file has the correct extantion
echo "Give the filename with the correct extantion:"
read filename
if [ "${filename: -4}" != ".log" ];
then
echo "Wrong File Argument"
echo "Instead use the correct form $filename.log"
exit 1
fi
```

Τέλος, ελέγχουμε αν το αρχείο που δόθηκε ως όρισμα από τον χρήστη βρίσκεται στο σύστημα. Συγκεκριμένα, μέσω της εντολής if [! -f "\$1"]; ελέγχουμε αν το πρώτο όρισμα που δόθηκε στο command line (-f είναι ένα flag που ελέγχει αν το αρχείο που δίνεται ως όρισμα είναι αρχείο, αντί για directory) δεν είναι ένα αρχείο.

Εάν η σύγκριση είναι αληθής (το αρχείο δεν υπάρχει), τότε :

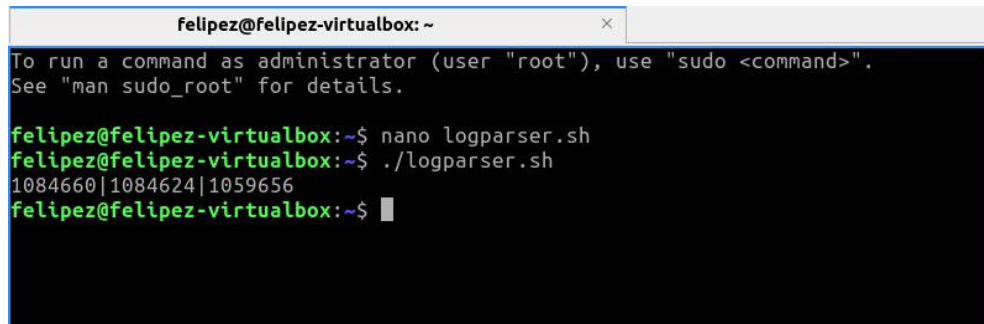
- ✓ Εκτελείται ο κώδικας μέσα στο if statement όπου εμφανίζεται το μήνυμα "Error! \$filename.log file not found." και στη συνέχεια εκτελείται η εντολή exit 1 η οποία διακόπτει την εκτέλεση του script.

Εάν όμως το αρχείο που δόθηκε ως όρισμα υπάρχει, τότε:

- ✓ Η εκτέλεση του script συνεχίζεται στις επόμενες γραμμές του κώδικα.

```
#Check if the log file exists
if [ ! -f "$1" ];
then
echo "Error! $filename.log file not found."
exit 1
fi
```

➤ Screenshot τερματικού για το ερώτημα i:

A screenshot of a terminal window titled 'felipez@felipez-virtualbox: ~'. The terminal shows a message about running commands as administrator. Then, the user runs 'nano logparser.sh' and './logparser.sh'. The output of the script is '1084660|1084624|1059656'.

```
felipez@felipez-virtualbox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

felipez@felipez-virtualbox:~$ nano logparser.sh
felipez@felipez-virtualbox:~$ ./logparser.sh
1084660|1084624|1059656
felipez@felipez-virtualbox:~$ █
```

Ερώτημα ii

Αρχικά , διαχωρίζουμε τις γραμμές του αρχείου που δόθηκε ως όρισμα στο command line χρησιμοποιώντας Internal Field Separator ως ένα newline character.¹

```
#Set the internal field separator to a newline character
IFS=$'\n'
```

Στη συνέχεια, ελέγχουμε αν το δεύτερο όρισμα που δόθηκε στο command line είναι κενό.

Εάν είναι κενό, τότε :

- ✓ Εκτελείται η εντολή `while read -r line; do echo "$line" done < "$1"` ². Κάθε γραμμή που διαβάζεται από το αρχείο εμφανίζεται στην οθόνη με την εντολή `echo "$line"`.

Αν δεν δοθεί κανένα δεύτερο όρισμα στο command line , (δηλαδή μόνο το `./logparser.sh access.log`) ,τότε θα εμφανίζονται όλες οι γραμμές του αρχείου στην οθόνη.

¹ Αυτό σημαίνει ότι κάθε νέα γραμμή του αρχείου θα θεωρείται σαν ένα ξεχωριστό πεδίο.

² Η εντολή `while read -r line; do echo "$line" done < "$1"` διαβάζει το αρχείο γραμμή-γραμμή χρησιμοποιώντας τον εσωτερικό διαχωριστή πεδίων που ορίστηκε παραπάνω.

```
if [ -z "$2" ]; then
while read -r line; do
echo "$line"
done < "$1"
fi
```

➤ Screenshot τερματικού για το ερώτημα ii:

Σε κάθε περίπτωση αν ο χρήστης δεν δώσει την σωστή προέκταση του αρχείου , εμφανίζεται μήνυμα σφάλματος:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log
Give the filename with the correct extantion:
file
Wrong File Argument
Instead use the correct form file.log
felipez@felipez-virtualbox:~$
```

Όταν ο χρήστης δώσει το αρχείο με την σωστή προέκταση έχουμε το παρακάτω στιγμιότυπο:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log
Give the filename with the correct extantion:
file
Wrong File Argument
Instead use the correct form file.log
felipez@felipez-virtualbox:~$ ./logparser.sh access.log
Give the filename with the correct extantion:
access.log
```

Εάν έχει δοθεί μόνο το “./logparser.sh access.log” , εμφανίζεται το περιεχόμενο του αρχείου ανα γραμμές :

```
felipez@felipez-virtualbox: ~
/1.1" 200 1659
::1 - - [13/Oct/2022:13:42:16 +0300] "POST /phpmyadmin/index.php?route=/ HTTP
/1.1" 200 1659
::1 - - [13/Oct/2022:13:53:49 +0300] "POST /phpmyadmin/index.php?route=/ HTTP
/1.1" 200 1659
::1 - - [13/Oct/2022:14:05:12 +0300] "POST /phpmyadmin/index.php?route=/ HTTP
/1.1" 200 1659
::1 - - [13/Oct/2022:14:16:24 +0300] "POST /phpmyadmin/index.php?route=/ HTTP
/1.1" 200 1659
::1 - - [13/Oct/2022:14:27:24 +0300] "POST /phpmyadmin/index.php?route=/ HTTP
/1.1" 200 1659
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /MyDocs/ HTTP/1.1" 200 3948
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/blank.gif HTTP/1.1" 200 148
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/back.gif HTTP/1.1" 200 216
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/folder.gif HTTP/1.1" 200 225
::1 - - [15/Oct/2022:19:27:57 +0300] "GET /MyDocs/Vathmoi/ HTTP/1.1" 200 517
::1 - - [15/Oct/2022:19:27:59 +0300] "GET /MyDocs/ HTTP/1.1" 200 3948
::1 - - [15/Oct/2022:19:28:04 +0300] "GET /MyDocs/Software HTTP/1.1" 301 350
::1 - - [15/Oct/2022:19:28:04 +0300] "GET /MyDocs/Software/ HTTP/1.1" 200 519
::1 - - [15/Oct/2022:19:28:08 +0300] "GET /MyDocs/Software/after.php HTTP/1.1
" 200 393
::1 - - [15/Oct/2022:19:28:10 +0300] "GET /MyDocs/Software/ HTTP/1.1" 200 519
felipez@felipez-virtualbox:~$
```

Ερώτημα iii

1° βήμα:

Αρχικά, χρησιμοποιούμε την συνάρτηση `mining_usernames` για να μετρήσουμε τον αριθμό των εγγραφών που υπάρχουν για κάθε χρήστη στο αρχείο. Η εντολή `awk` είναι αρκετά βοηθητική στην ανάλυση του αρχείου, ώστε να καταγράψει τον ακριβή αριθμό των εγγραφών για κάθε χρήστη σε έναν πίνακα `user_counts`. Στη συνέχεια, στο `END` εκτυπώνεται ο αριθμός των εγγραφών και το όνομα του χρήστη για κάθε χρήστη στον πίνακα `user_counts`. Τέλος, η εντολή `sort -k 2` χρησιμοποιείται για να ταξινομήσει τα αποτελέσματα ανάλογα με το όνομα του χρήστη.

```
#Define the mining_usernames funcion
mining_usernames() {
awk '{ user_counts[$3]++ } END {
for(user in user_counts) {
printf "%d %s\n", user_counts[user],user
}
}' "$1" | sort -k 2
}
```

2° βήμα:

Ελέγχουμε αν έχει παραχωρηθεί η επιλογή στην γραμμή εντολών `--usrid` και αν ναι, ελέγχουμε αν έχει παραχωρηθεί και ένας ρόλος χρήστη.

Εάν δεν έχει παραχωρηθεί κανένας ρόλος χρήστη, τότε:

- ✓ Καλείται η συνάρτηση `"mining_usernames"` εμφανίζοντας το πλήθος και το όνομα του κάθε χρήστη.

Εάν έχει παραχωρηθεί ένας ρόλος χρήστη:

- ✓ χρησιμοποιούμε την εντολή `"sed"` για να εμφανίσουμε τις εγγραφές για αυτόν τον συγκεκριμένο ρόλο χρήστη (`root`, `admin`, `user1`, `user2`, `user3`, `president`) από το αρχείο `access.log`.

Αν ο ρόλος χρήστη δεν είναι έγκυρος, εμφανίζεται ένα μήνυμα σφάλματος.

```
#Check if the option --usrid was provided
if [ "$2" = "--usrid" ];
then
#Check if a user role was provided
if [ -z "$3" ];
then
#None of them provided, so display the counts for all the users
mining_usernames "$1"
else
```

```
#User role provided, so display the log entries and the information for the specific
user

case $3 in
root)

#Print the log entries for root
sed -n '/root/p' "$1"

;;

admin)

#Print the log entries for admin
sed -n '/admin/p' "$1"

;;

user1)

#Print the log entries for user1
sed -n '/user1/p' "$1"

;;

user2)

#Print the log entries for user2
sed -n '/user2/p' "$1"

;;

user3)

#Print the log entries for user3
sed -n '/user3/p' "$1"

;;

president)

#Print the log entries for president
sed -n '/president/p' "$1"

;;

*)

#Invalid user role
echo "ERROR: Invalid user role!"
exit 1

;;

esac

fi

# --usrid option not provided, so display the contents of the log file line by line
fi
```

➤ Screenshot τερματικού για το ερώτημα iii:

Σε κάθε περίπτωση αν ο χρήστης δεν δώσει την σωστή προέκταση του αρχείου , εμφανίζεται μήνυμα σφάλματος:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid
Give the filename with the correct extantion:
fi
Wrong File Argument
Instead use the correct form fi.log
felipez@felipez-virtualbox:~$
```

Εάν έχει δοθεί μόνο το “./logparser.sh access.log --usrid” , εμφανίζεται το πλήθος του κάθε ρόλου του αρχείου:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid
Give the filename with the correct extantion:
fi
Wrong File Argument
Instead use the correct form fi.log
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid
Give the filename with the correct extantion:
access.log
15710 -
124 admin
34 president
181 root
110 user1
91 user2
39 user3
felipez@felipez-virtualbox:~$
```

Εάν δεν θάλουμε σωστό ρόλο χρήστη εμφανίζεται το παρακάτω μήνυμα σφάλματος:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid a
Give the filename with the correct extantion:
access.log
ERROR: Invalid user role!
felipez@felipez-virtualbox:~$
```

Επιλέγουμε τον ρόλο χρήστη user3:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid user3
```

Δίνουμε την κατάλληλη προέκταση του αρχείου:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --usrid user3
Give the filename with the correct extantion:
access.log
```


Εμφανίζονται τα περιεχόμενα του αρχείου που αφορούν τον ρόλο χρήστη user3:

```
felipez@felipez-virtualbox: ~  
mg/b_newdb.png HTTP/1.1" 200 585  
::1 - user3 - [11/Feb/2022:15:52:30 +0300] "GET /phpmyadmin/themes/pmahomme/i  
mg/console.png HTTP/1.1" 200 292  
::1 - user3 - [14/Feb/2022:15:59:13 +0300] "GET /phpmyadmin/js/dist/gis_data_  
editor.js?v=5.1.1 HTTP/1.1" 200 13822  
::1 - user3 - [19/Feb/2022:16:14:07 +0300] "GET /MyDocs/AEKX/login.php HTTP/1  
.1" 302 -  
::1 - user3 - [19/Feb/2022:16:14:14 +0300] "GET /MyDocs/AEKX/login.php HTTP/1  
.1" 302 -  
::1 - user3 - [19/Feb/2022:16:14:14 +0300] "GET /MyDocs/AEKX/login.php HTTP/1  
.1" 302 -  
::1 - user3 - [21/Feb/2022:16:14:14 +0300] "GET /MyDocs/AEKX/login.php HTTP/1  
.1" 302 -  
::1 - user3 - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1  
.1" 302 -  
127.0.0.1 - user3 - [29/Mar/2022:17:42:47 +0300] "POST /MyDocs/AEKX/chiss.php  
HTTP/1.1" 200 783 "http://localhost/MyDocs/AEKX/inscomplaints.php" "Mozilla/  
5.0 (Windows NT 10.0; Win64; x64; rv:101.0) Gecko/20100101 Firefox/101.0"  
::1 - user3 - [08/Jun/2022:17:59:24 +0300] "GET /phpmyadmin/index.php?route=  
sql&db=aekx&table=complaints&pos=0 HTTP/1.1" 200 19413  
::1 - user3 - [09/Jun/2022:06:08:50 +0300] "GET /phpmyadmin/js/messages.php?l  
=en&v=5.1.1 HTTP/1.1" 200 7921  
felipez@felipez-virtualbox:~$
```

Ερώτημα iv:

Ελέγχουμε αν η επιλογή -method έχει δοθεί ως παράμετρος στην γραμμή εντολών.

Αν δεν δόθηκε κάποιο όνομα μεθόδου:

- ✓ Εμφανίζεται ένα μήνυμα σφάλματος το οποίο προειδοποιεί τον χρήστη να δώσει ένα όνομα μεθόδου. Αν δόθηκε κάποιο όνομα μεθόδου, ελέγχεται αν είναι έγκυρο (GET ή POST).
- ✓

Σε περίπτωση που είναι έγκυρο:

- ✓ Εμφανίζονται οι εγγραφές του αρχείου που περιέχουν το όνομα της συγκεκριμένης μεθόδου.

Σε περίπτωση που το όνομα της μεθόδου δεν είναι έγκυρο:

- ✓ Εμφανίζεται μήνυμα σφάλματος, δηλαδή ότι πρέπει ο χρήστης να δώσει μια από τις δύο επιλογές (GET ή POST).

```
#Check if the option -method was provided  
if [ "$2" = "-method" ]; then  
#Check if a method name was provided  
if [ -z "$3" ]; then  
echo "Error: You should give a method name!"  
exit 1  
fi  
#Check if method name is valid  
case "$3" in  
GET)  
sed -n '/GET/p' "$1"
```

```
;;
POST)
sed -n '/POST/p' "$1"
;;
*)
echo "Error: You should give a correct method name! (Must be GET or POST)"
exit 1
;;
esac
fi
```

➤ Screenshot τερματικού για το ερώτημα iv:

Εάν έχει δοθεί μόνο το “./logparser.sh access.log -method”

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method
```

Θα μας εμφανίσει το παρακάτω μήνυμα σφάλματος καθώς θα έπρεπε να έχουμε εισάγει και το όνομα της μεθόδου:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method
Give the filename with the correct extantion:
access.log
Error: You should give a method name!
felipez@felipez-virtualbox:~$
```

Αν δεν γράψουμε το κατάλληλο όνομα της μεθόδου (GET or POST) μας εμφανίζει το παρακάτω μήνυμα σφάλματος:

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method
Give the filename with the correct extantion:
access.log
Error: You should give a method name!
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method GP
Give the filename with the correct extantion:
access.log
Error: You should give a correct method name! (Must be GET or POST)
felipez@felipez-virtualbox:~$
```

Επιλέγουμε την μέθοδο GET

```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method
Give the filename with the correct extantion:
access.log
Error: You should give a method name!
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method GP
Give the filename with the correct extantion:
access.log
Error: You should give a correct method name! (Must be GET or POST)
felipez@felipez-virtualbox:~$ ./logparser.sh access.log -method GET
Give the filename with the correct extantion:
access.log
```

Εμφανίζονται τα περιεχόμενα του αρχείου που αφορούν την μέθοδο GET:

```
felipez@felipez-virtualbox: ~  
99  
::1 - - [13/Oct/2022:03:22:53 +0300] "GET /phpmyadmin/index.php?route=/recent  
-table&ajax_request=1&recent_table=1&no_debug=true&_nocache=16656205731393815  
01&token=32567d5d795a3d6d36285a4b68702655 HTTP/1.1" 200 1754  
::1 - - [13/Oct/2022:03:23:21 +0300] "GET /phpmyadmin/index.php?route=/table/  
sql&db=sweng&table=currprojs&ajax_request=true&ajax_page_request=true&_nocach  
e=1665620601252531786&token=32567d5d795a3d6d36285a4b68702655 HTTP/1.1" 200 51  
99  
::1 - - [13/Oct/2022:03:23:21 +0300] "GET /phpmyadmin/index.php?route=/recent  
-table&ajax_request=1&recent_table=1&no_debug=true&_nocache=16656206014829999  
79&token=32567d5d795a3d6d36285a4b68702655 HTTP/1.1" 200 1754  
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /MyDocs/ HTTP/1.1" 200 3948  
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/blank.gif HTTP/1.1" 200 148  
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/back.gif HTTP/1.1" 200 216  
::1 - - [15/Oct/2022:19:27:55 +0300] "GET /icons/folder.gif HTTP/1.1" 200 225  
::1 - - [15/Oct/2022:19:27:57 +0300] "GET /MyDocs/Vathmoi/ HTTP/1.1" 200 517  
::1 - - [15/Oct/2022:19:27:59 +0300] "GET /MyDocs/ HTTP/1.1" 200 3948  
::1 - - [15/Oct/2022:19:28:04 +0300] "GET /MyDocs/Software HTTP/1.1" 301 350  
::1 - - [15/Oct/2022:19:28:04 +0300] "GET /MyDocs/Software/ HTTP/1.1" 200 519  
::1 - - [15/Oct/2022:19:28:08 +0300] "GET /MyDocs/Software/after.php HTTP/1.1  
" 200 393  
::1 - - [15/Oct/2022:19:28:10 +0300] "GET /MyDocs/Software/ HTTP/1.1" 200 519  
felipez@felipez-virtualbox:~$
```

Ερώτημα ν:

Ελέγχουμε αν η επιλογή --servprot δόθηκε ως παράμετρος στην γραμμή εντολών

Αν έχει δοθεί, τότε:

- ✓ Ελέγχει αν δόθηκε και ένα πρωτόκολλο δικτύου.

Αν δεν έχει δοθεί:

- ✓ Εμφανίζεται ένα μήνυμα σφάλματος που ζητά από τον χρήστη να δώσει ένα πρωτόκολλο δικτύου.

Αν έχει δοθεί η παράμετρος:

- ✓ Ελέγχει την εγκυρότητά της (IPv4 ή IPv6) και εμφανίζει ανάλογα τις εγγραφές του αρχείου για αυτό το πρωτόκολλο.

```
#Check if the option --servprot was provided  
if [ "$2" = "--servprot" ]; then  
#Check if a network protocol was provided  
if [ -z "$3" ]; then  
echo "Error: You should give a Network Protocol!"  
exit 1  
fi  
#Check if Network Protocol is valid  
case "$3" in  
IPv4)  
sed -n '/127.0.0.1/p' "$1"  
;;  
IPv6)
```

```
sed -n '/:::1/p' "$1"

;;

*)

echo "Error: You should give a correct Network Protocol! (Must be IPv4 or IPv6)"

exit 1

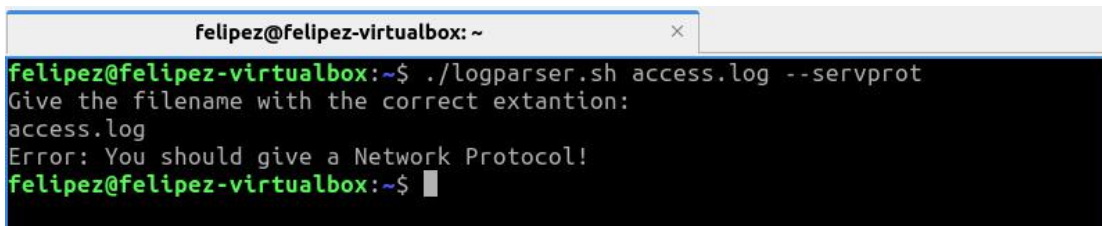
;;

Esac

fi
```

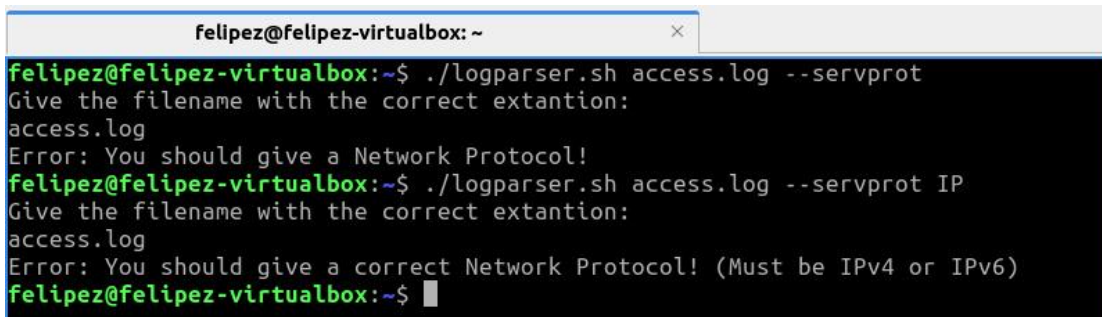
➤ Screenshot τερματικού για το ερώτημα ν:

Εάν έχει δοθεί μόνο το `./logparser.sh access.log --servprot`, εμφανίζεται μήνυμα σφάλματος καθώς δεν έχουμε δώσει κάποιο πρωτόκολλο:



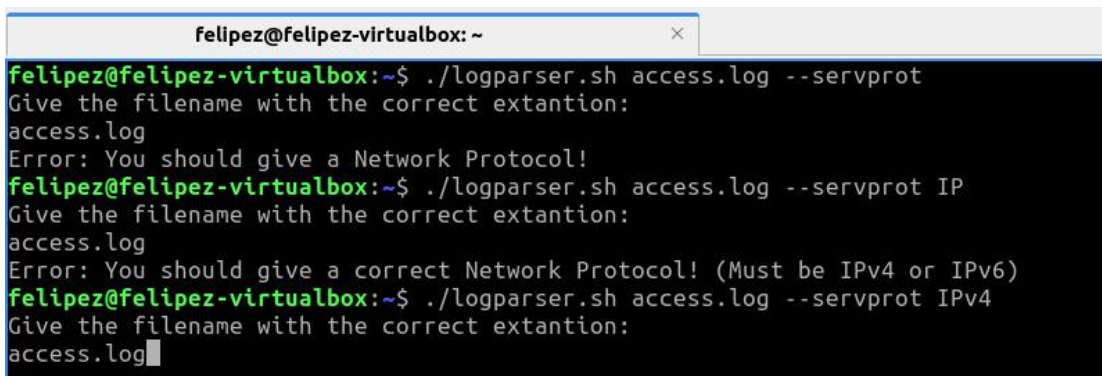
```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot
Give the filename with the correct extantion:
access.log
Error: You should give a Network Protocol!
felipez@felipez-virtualbox:~$
```

Αν εισάγουμε λάθος όνομα πρωτοκόλλου εμφανίζεται μήνυμα σφάλματος:



```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot
Give the filename with the correct extantion:
access.log
Error: You should give a Network Protocol!
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot IP
Give the filename with the correct extantion:
access.log
Error: You should give a correct Network Protocol! (Must be IPv4 or IPv6)
felipez@felipez-virtualbox:~$
```

Επιλέγουμε το πρωτόκολλο IPv4:



```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot
Give the filename with the correct extantion:
access.log
Error: You should give a Network Protocol!
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot IP
Give the filename with the correct extantion:
access.log
Error: You should give a correct Network Protocol! (Must be IPv4 or IPv6)
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --servprot IPv4
Give the filename with the correct extantion:
access.log
```


Εμφάνιση περιεχομένου που σχετίζεται με το πρωτοκολλο IPv4 το οποίο αντιστοιχεί με το 127.0.0.1:

```
felipez@felipez-virtualbox: ~
127.0.0.1 - - [20/Jul/2022:19:07:28 +0300] "POST /MyDocs/AEKX/en/resusearccrc
.php HTTP/1.1" 200 18862 "http://localhost/MyDocs/AEKX/en/searchsinglecrinde
xes.php" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:102.0) Gecko/20100101
Firefox/102.0"
127.0.0.1 - - [02/Oct/2022:12:28:52 +0300] "GET /MyDocs/IPEM/ HTTP/1.1" 200 3
587 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 F
irefox/105.0"
127.0.0.1 - - [02/Oct/2022:12:28:52 +0300] "GET /MyDocs/IPEM/images/favicon-3
2x32.png HTTP/1.1" 404 296 "http://localhost/MyDocs/IPEM/" "Mozilla/5.0 (Wind
ows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 Firefox/105.0"
127.0.0.1 - - [02/Oct/2022:12:28:52 +0300] "GET /MyDocs/IPEM/images/favicon-1
6x16.png HTTP/1.1" 404 296 "http://localhost/MyDocs/IPEM/" "Mozilla/5.0 (Wind
ows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 Firefox/105.0"
127.0.0.1 - - [02/Oct/2022:12:37:21 +0300] "GET /MyDocs/IPEM/ HTTP/1.1" 200 4
073 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 F
irefox/105.0"
127.0.0.1 - - [02/Oct/2022:12:37:21 +0300] "GET /MyDocs/IPEM/images/logo-head
er.png HTTP/1.1" 200 22434 "http://localhost/MyDocs/IPEM/" "Mozilla/5.0 (Wind
ows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 Firefox/105.0"
127.0.0.1 - - [02/Oct/2022:12:37:21 +0300] "GET /MyDocs/IPEM/images/favicon-1
6x16.png HTTP/1.1" 200 958 "http://localhost/MyDocs/IPEM/" "Mozilla/5.0 (Wind
ows NT 10.0; Win64; x64; rv:105.0) Gecko/20100101 Firefox/105.0"
felipez@felipez-virtualbox:~$
```

Ερώτημα vi:

Δημιουργούμε την συνάρτηση `count_browsers()` και με τη βοήθεια της `awk` καταμετρούμε τους browsers που χρησιμοποιούνται στο αρχείο και τα ταξινομούμε με βάση το πλήθος τους. Συγκεκριμένα, το `awk` ελέγχει κάθε γραμμή του αρχείου για εμφάνιση "Mozilla", "Chrome", "Safari" και "Edg" και αυξάνει τον μετρητή για αυτόν τον browser στο `browser_counts`. Στο τέλος εκτυπώνει τα αποτελέσματα για κάθε browser σε αύξουσα σειρά.

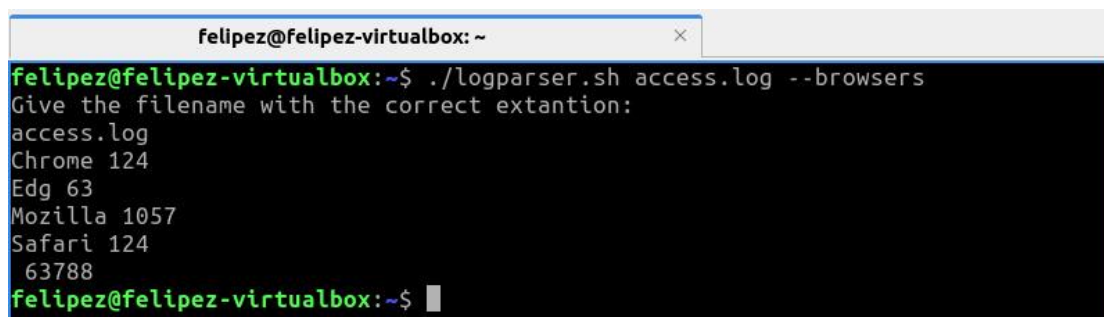
```
#Define the count_browsers function
count_browsers() {
awk '{ match($0, "Mozilla")
browser_counts[substr($0, RSTART, RLENGTH)]++
match($0, "Chrome")
browser_counts[substr($0, RSTART, RLENGTH)]++
match($0, "Safari")
browser_counts[substr($0, RSTART, RLENGTH)]++
match($0, "Edg")
browser_counts[substr($0, RSTART, RLENGTH)]++
}
END {
for(browser in browser_counts) {
printf "%s %d\n", browser,browser_counts[browser]
}
}' "$1" | sort -n
}
```

Επίσης , ελέγχουμε αν δόθηκε η επιλογή --browsers στη γραμμή εντολών. Εάν έχει δοθεί, εκτυπώνουμε τους πίνακες μετρήσεων για όλους τους περιηγητές και τερματίζεται το script.

```
#Check if the option --browsers was provided
if [ "$2" = "--browsers" ]; then
#Display the counts for all the browsers
count_browsers "$1"
exit 1
fi
```

➤ **Screenshot τερματικού για το ερώτημα vi:**

Εάν έχει δοθεί μόνο το `./logparser.sh access.log --browsers` , εμφανίζεται το πλήθος των διαφορετικών φυλλομετρητών:



```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --browsers
Give the filename with the correct extantion:
access.log
Chrome 124
Edg 63
Mozilla 1057
Safari 124
63788
felipez@felipez-virtualbox:~$
```

Ερώτημα vii:

Αρχικά ελέγχουμε αν ο χρήστης έχει δώσει την επιλογή "--datum" ως δεύτερο όρισμα στην γραμμή εντολών. Αν ναι, τότε η μεταβλητή "datum" παίρνει την τιμή του τρίτου ορίσματος. Στη συνέχεια, ελέγχεται αν η τιμή της "datum" αντιστοιχεί σε έναν μήνα (Jan, Feb, Mar κλπ) και έπειτα δημιουργείται ένας case switch όπου για κάθε μήνα που επιλέγεται, εκτελείται αντίστοιχα ένα sed -n '/μήνας/p' "\$1" για να επιστρέψει τις εγγραφές του αρχείου για αυτόν τον μήνα ανεξαρτήτως χρόνου. Δώσαμε την δυνατότητα ο χρήστης να δίνει τον μήνα άφοβα στην γραμμή εντολών (χωρίς να τον απασχολεί αν είναι case – sensitive , δηλαδή είτε γράψει FeB είτε fEb ,είναι ακριβώς το ίδιο και θα επιστραφούν οι εγγραφές του αρχείου για τον μήνα Φεβρουάριο) ανεξαρτήτως χρόνου.

```
#Check if the option --datum was provided
if [ "$2" = "--datum" ];
then
datum=$3
#Check if date is a month
if [[ "$datum" =~ ^[Jj][Aa][Nn]$ ]]; then
datum="Jan"
elif [[ "$datum" =~ ^[Ff][Ee][Bb]$ ]]; then
datum="Feb"
elif [[ "$datum" =~ ^[Mm][Aa][Rr]$ ]]; then
datum="Mar"
elif [[ "$datum" =~ ^[Aa][Pp][Rr]$ ]]; then
```

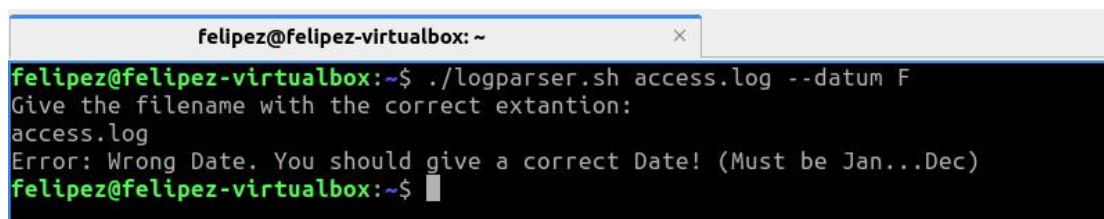
```
datum="Apr"
elif [[ "$datum" =~ ^[Mm][Aa][Yy]$ ]]; then
datum="May"
elif [[ "$datum" =~ ^[Jj][Uu][Nn]$ ]]; then
datum="Jun"
elif [[ "$datum" =~ ^[Jj][Uu][Ll]$ ]]; then
datum="Jul"
elif [[ "$datum" =~ ^[Aa][Uu][Gg]$ ]]; then
datum="Aug"
elif [[ "$datum" =~ ^[Ss][Ee][Pp]$ ]]; then
datum="Sep"
elif [[ "$datum" =~ ^[Oo][Cc][Tt]$ ]]; then
datum="Oct"
elif [[ "$datum" =~ ^[Nn][Oo][Vv]$ ]]; then
datum="Nov"
elif [[ "$datum" =~ ^[Dd][Ee][Cc]$ ]]; then
datum="Dec"
else
echo "Error: Wrong Date. You should give a correct Date! (Must be Jan...Dec)"
exit 1
fi

case "$datum" in
Jan)
sed -n '/Jan/p' "$1"
;;
Feb)
sed -n '/Feb/p' "$1"
;;
Mar)
sed -n '/Mar/p' "$1"
;;
Apr)
sed -n '/Apr/p' "$1"
;;
May)
```

```
sed -n '/May/p' "$1"
;;
Jun)
sed -n '/Jun/p' "$1"
;;
Jul)
sed -n '/Jul/p' "$1"
;;
Aug)
sed -n '/Aug/p' "$1"
;;
Sep)
sed -n '/Sep/p' "$1"
;;
Oct)
sed -n '/Oct/p' "$1"
;;
Nov)
sed -n '/Nov/p' "$1"
;;
Dec)
sed -n '/Dec/p' "$1"
;;
*)
exit 1
esac
fi
```

➤ Screenshot τερματικού για το ερώτημα vii:

Εάν έχει δοθεί το `./logparser.sh access.log --datum` και ένας λανθασμένος μήνας εμφανίζεται μήνυμα σφάλματος:



```
felipez@felipez-virtualbox: ~
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --datum F
Give the filename with the correct extantion:
access.log
Error: Wrong Date. You should give a correct Date! (Must be Jan...Dec)
felipez@felipez-virtualbox:~$
```


Επιλέγουμε τον μήνα Φεβρουάριο:

```
felipez@felipez-virtualbox: ~  
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --datum F  
Give the filename with the correct extantion:  
access.log  
Error: Wrong Date. You should give a correct Date! (Must be Jan...Dec)  
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --datum fEb  
Give the filename with the correct extantion:  
access.log
```

Εμφανίζονται τα περιεχόμενα του αρχείου για τον μήνα Φεβρουάριο ανεξαρτήτως χρονολογίας:

```
felipez@felipez-virtualbox: ~  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - root - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - admin - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - president - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - user3 - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
felipez@felipez-virtualbox:~$
```

Δείχνουμε ότι με την υλοποίησή μας έχουμε κάνει *case sensitive* την επιλογή του μήνα:

```
felipez@felipez-virtualbox: ~  
::1 - root - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - admin - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - president - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - user3 - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
felipez@felipez-virtualbox:~$ ./logparser.sh access.log --datum FeB  
Give the filename with the correct extantion:  
access.log
```

```
felipez@felipez-virtualbox: ~  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - root - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - admin - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - president - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - user3 - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
::1 - - [21/Feb/2022:16:14:15 +0300] "GET /MyDocs/AEKX/login.php HTTP/1.1" 302 -  
felipez@felipez-virtualbox:~$
```

Τέλος ,παραθέτουμε τα στιγμιότυπα του κώδικα που δημιουργούμε το αρχείο logparser.sh :

```
felipez@felipez-virtualbox: ~  
GNU nano 6.2 logparser.sh  
#!/bin/bash  
  
#Team Members  
  
#Filippos-Paraskevas Zygouris  
std1AM="1084660"  
#Niki-Aikaterini Kyriakatos  
std2AM="1084624"  
#Maria-Anastasia Kyriakatos  
std3AM="1059656"  
  
#Check if a file was provided as an argument in the command line  
if [ $# -eq 0 ];  
then  
#No file provided ,so display team members IDs  
echo "$std1AM|$std2AM|$std3AM"  
exit 0  
fi  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify  
  
felipez@felipez-virtualbox:~$ nano logparser.sh
```

```
felipez@felipez-virtualbox: ~  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
felipez@felipez-virtualbox:~$ nano logparser.sh
```

2^η ΆΣΚΗΣΗ

Για να εφαρμόσουμε τον κώδικα που μας δόθηκε χρησιμοποιώντας πολλαπλές διεργασίες και ουρές μηνυμάτων, θα χρειαστεί να χρησιμοποιήσουμε μηχανισμούς επικοινωνίας μεταξύ διεργασιών, όπως `fork()` και `msgget()`. Για να το πετύχουμε αυτό ακολουθούμε τα εξής βήματα:

Βήμα 1^ο:

Δημιουργούμε μια ουρά μηνυμάτων μέσω της `msgget()` και ορίζουμε μια δομή για το μήνυμα. Το μήνυμα περιέχει τον αριθμό των διεργασιών, το διάστημα για κάθε διεργασία και το αποτέλεσμα του ολοκληρώματος για κάθε διεργασία.

Βήμα 2^ο:

Με την βοήθεια της `fork()`, δημιουργούμε τον επιθυμητό αριθμό θυγατρικών διεργασιών (παιδιών).

Βήμα 3^ο:

Στις θυγατρικές διεργασίες, υπολογίζουμε το ολοκλήρωμα με τη χρήση του δοσμένου τύπου και το διάστημα που έχει εκχωρηθεί σε κάθε διεργασία.

Βήμα 4^ο:

Χρησιμοποιούμε την `msgsnd()` για να στείλουμε το αποτέλεσμα του ολοκληρώματος στη γονική διαδικασία.

Βήμα 5^ο:

Στη γονική διαδικασία, χρησιμοποιούμε τη `msgrcv()` με σκοπό να λάβουμε τα αποτελέσματα από τις θυγατρικές διεργασίες.

Βήμα 6^ο:

Αφού λάβουμε όλα τα αποτελέσματα, τα προσθέτουμε και εκτυπώνουμε το τελικό αποτέλεσμα.

Βήμα 7^ο:

Όταν η γονική διαδικασία λάβει μηνύματα από όλες τις θυγατρικές διεργασίες, θα έχει το τελικό αποτέλεσμα του ολοκληρώματος. Στη συνέχεια, μπορεί να εκτυπώσει το αποτέλεσμα και τον χρόνο που έχει παρέλθει χρησιμοποιώντας τη συνάρτηση `get_wtime`.

Βήμα 8^ο:

Χρησιμοποιούμε το `msgctl()` για να διαγράψουμε την ουρά μηνυμάτων.

Ακολουθεί ο κώδικας που δείχνει πώς να χρησιμοποιούμε την `fork()` και τις ουρές μηνυμάτων για επικοινωνία μεταξύ διεργασιών μαζί με επεξήγηση. Αρχικά γίνεται χρήση των header files : `stdio.h`³, `stdlib.h`⁴, `unistd.h`⁵, `sys/types.h`⁶, `sys/ipc.h`⁷, `sys/msg.h`⁸ και `math.h`⁹ . Η συμπερίληψη αυτών των αρχείων κεφαλίδας επιτρέπει στο πρόγραμμα να χρησιμοποιεί τις λειτουργίες και τους τύπους δεδομένων που ορίζονται σε αυτά.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <math.h>
#include <sys/time.h>
```

Έπειτα, ορίζουμε έναν τύπο δομής που ονομάζεται μήνυμα ο οποίος χρησιμοποιείται για την αποθήκευση δεδομένων σε μια ουρά μηνυμάτων. Η ουρά μηνυμάτων επιτρέπει στις διεργασίες να επικοινωνούν μεταξύ τους στέλνοντας και λαμβάνοντας μηνύματα.

Διαθέτει δύο πεδία:

- 1) **type**: μια μεγάλη ακέραια τιμή που μπορεί να χρησιμοποιηθεί για την ταξινόμηση των μηνυμάτων και τον προσδιορισμό της διεργασίας που θα τα λάβει.
- 2) **result**: μια τιμή κινητής υποδιαστολής διπλής ακρίβειας που μπορεί να χρησιμοποιηθεί για την αποθήκευση του αποτελέσματος ενός υπολογισμού, που πρέπει να περάσουν μεταξύ των διεργασιών.

Το πεδίο `type` μπορεί να χρησιμοποιηθεί για τον καθορισμό ενός συγκεκριμένου τύπου μηνύματος και η συνάρτηση `msgrcv`¹⁰ μπορεί να ενημερωθεί ότι λαμβάνει μόνο μηνύματα συγκεκριμένου τύπου(`type`).

Αυτό μας προφυλάσσει και στην περίπτωση που η ουρά μηνυμάτων χρησιμοποιείται για την επικοινωνία πολλών τύπων δεδομένων μεταξύ διεργασιών.

³ **stdio.h**: Ορίζει λειτουργίες και μακροεντολές για την εκτέλεση λειτουργιών εισόδου και εξόδου στο C, όπως η ανάγνωση από και η εγγραφή σε αρχεία και η ανάγνωση και εγγραφή δεδομένων από και προς τις τυπικές ροές εισόδου και εξόδου (π.χ. το πληκτρολόγιο). Μερικές από τις κοινές λειτουργίες και μακροεντολές που ορίζονται στο `stdio.h` περιλαμβάνουν `printf`, `scanf`, `fopen`, `fclose`, `fread` και `fwrite`.

⁴ **stdlib.h**: Ορίζει συναρτήσεις και μακροεντολές για την εκτέλεση λειτουργιών βοηθητικού προγράμματος γενικής χρήσης, όπως η εκχώρηση μνήμης, η μετατροπή συμβολοσειρών και η δημιουργία ψευδοτυχαίων αριθμών. Μερικές από τις κοινές συναρτήσεις και μακροεντολές που ορίζονται στο `stdlib.h` περιλαμβάνουν `malloc`, `free`, `atoi`, `atof` και `rand`.

⁵ **unistd.h**: Ορίζει διάφορες λειτουργίες για την εκτέλεση βασικών κλήσεων συστήματος, όπως `fork`, `exec`, `sleep` και `getpid`.

⁶ **sys/types.h**: Ορίζει διάφορους τύπους δεδομένων που χρησιμοποιούνται σε κλήσεις συστήματος και σε άλλα μέρη του λειτουργικού συστήματος.

⁷ **sys/ipc.h**: Ορίζει σταθερές και δομές δεδομένων που χρησιμοποιούνται σε λειτουργίες επικοινωνίας μεταξύ διεργασιών (IPC), όπως ουρές μηνυμάτων και σηματοφόρους.

⁸ **sys/msg.h**: Ορίζει σταθερές και δομές δεδομένων που χρησιμοποιούνται σε συναρτήσεις ουράς μηνυμάτων, όπως `msgget`, `msgsnd` και `msgrcv`.

⁹ **math.h**: Ορίζει διάφορες μαθηματικές συναρτήσεις και μακροεντολές, όπως `sqrt`, `pow` και `M_PI`.

¹⁰ **msgrcv** : Χρησιμοποιείται για τη λήψη μηνυμάτων από μια ουρά μηνυμάτων

```
// Structure for message queue
struct message {
    long type;
    double result;
};
```

Η συνάρτηση `main` διαθέτει 2 παραμέτρους, τις `argc`¹¹ και `argv`¹². Διαβάζουμε τον αριθμό των διεργασιών από τη γραμμή εντολών ή χρησιμοποιούμε προεπιλεγμένη(default) τιμή. Επιπλέον, Χρησιμοποιούμε τον τελεστή `?` `:`, για να προσδιορίσουμε την τιμή της μεταβλητής `num_processes`. Παίρνει 3 τελεστές, ο πρώτος τελεστής είναι `Boolean`, ο δεύτερος και ο τρίτος τελεστής είναι εκφράσεις που αξιολογούνται με βάση την τιμή της `Boolean` μεταβλητής. Έπειτα, χρησιμοποιούμε την δυαδική έκφραση στον κώδικά μας είναι `argc > 1`, η οποία ελέγχει εάν είναι μεγαλύτερος από 1 ο αριθμός των ορισμάτων της γραμμής εντολών που διαβιβάζονται στο πρόγραμμα. Εάν αυτή η έκφραση είναι αληθής, τότε η τιμή του `num_processes` ορίζεται σε `atoi(argv[1])`, που μετατρέπει τη συμβολοσειρά που είναι αποθηκευμένη στο `argv[1]` σε ακέραιο αριθμό. Η τιμή του `num_processes` ορίζεται σε 1, εάν η έκφραση `boolean` είναι `false`. Συνεπώς, η εντολή ελέγχει εάν ένα όρισμα της γραμμής εντολών μεταβιβάστηκε στο πρόγραμμα και αν ναι, ορίζει το `num_processes` στην τιμή αυτού του ορίσματος. Εάν δεν διαβιβάστηκε τίποτα από την γραμμή εντολών, τότε θέτει το `num_processes` σε 1, οπότε υπολογίζει 1 φορά την τιμή του ολοκληρώματος.

```
int main(int argc, char *argv[])
{
    int num_processes = argc > 1 ? atoi(argv[1]) : 1;
```

```
// Create message queue
key_t key = ftok("/tmp", 'a');
```

Χρησιμοποιούμε την συνάρτηση `ftok()`¹³ με όνομα διαδρομής¹⁴ `++ "/tmp"` και αναγνωριστικό έργου¹⁵ είναι `'a'`, που είναι ο κωδικός ASCII για τον χαρακτήρα `'a'`. Η `ftok()` θα δημιουργήσει ένα μοναδικό κλειδί με βάση τον συνδυασμό αυτών των δύο τιμών και θα το επιστρέψει ως τύπο `key_t`. Ο τύπος `key_t` είναι ένας `typedef` για έναν ακέραιο τύπο που χρησιμοποιείται για να αναπαραστήσει το κλειδί που δημιουργείται από το `ftok()`. Στη συνέχεια, αυτό το κλειδί χρησιμοποιείται στην κλήση στη `msgget()` για τη δημιουργία της ουράς μηνυμάτων.

```
int msgid = msgget(key, IPC_CREAT | 0666);
```

¹¹ **argc**: Μια ακέραια τιμή που υποδεικνύει τον αριθμό των ορισμάτων που μεταβιβάστηκαν στο πρόγραμμα κατά την εκτέλεσή του.

¹² **argv**: Ένας πίνακας συμβολοσειρών που περιέχει τα ορίσματα που μεταβιβάστηκαν στο πρόγραμμα κατά την εκτέλεσή του.

¹³ **ftok()**: Είναι μια συνάρτηση, η οποία δημιουργεί ένα μοναδικό κλειδί για μια ουρά μηνυμάτων, σηματοφόρο ή κοινόχρηστη μνήμη. Αυτό το κλειδί χρησιμοποιείται για την αναγνώριση της ουράς μηνυμάτων, του σηματοφόρου ή της κοινόχρηστης μνήμης σε επόμενες κλήσεις προς `msgget()`, `semget()` ή `shmget()`. Η συνάρτηση `ftok()` παίρνει δύο ορίσματα: ένα αναγνωριστικό έργου και ένα όνομα διαδρομής(pathname).

¹⁴ **Αναγνωριστικό έργου(project identifier)**: Είναι μια ακέραια τιμή που χρησιμοποιείται για να διακρίνει διαφορετικές ουρές μηνυμάτων, σηματοφόρους ή κοινόχρηστες μνήμες που χρησιμοποιούν το ίδιο όνομα διαδρομής.

¹⁵ **Όνομα διαδρομής**: Είναι μια συμβολοσειρά που καθορίζει τη διαδρομή προς ένα αρχείο που χρησιμοποιείται ως βάση για τη δημιουργία του κλειδιού.

Η σημαία `IPC_CREAT`¹⁶ με τιμή δικαιωμάτων 0666 λέει στη `msgget()`¹⁷ να δημιουργήσει την ουρά μηνυμάτων εάν δεν υπάρχει ήδη.

Εάν η `msgget()` επιτευχθεί, επιστρέφει ένα αναγνωριστικό ουράς μηνυμάτων ή `msgid`¹⁸. Εάν η `msgget()` είναι ανεπιτυχής, επιστρέφει -1.

Η `msgget()` δημιουργεί μια ουρά μηνυμάτων με το καθορισμένο κλειδί, δικαιώματα και επιστρέφει το αναγνωριστικό ουράς μηνυμάτων ως τύπο `integer`. Αυτό το αναγνωριστικό χρησιμοποιείται στη συνέχεια σε επόμενες κλήσεις προς άλλες συναρτήσεις ουράς μηνυμάτων για να αναγνωριστούν οι ουρές μηνυμάτων.

Έπειτα, ελέγχουμε εάν η τιμή του `msgid` είναι μικρότερη από 0, γεγονός που δείχνει ότι η συνάρτηση `msgget()` δεν επιτεύχθηκε. Εάν το `msgid` είναι μικρότερο από 0, τότε η συνάρτηση `perror()`¹⁹ καλείται με τη συμβολοσειρά "msgget" ως όρισμα. Για παράδειγμα, εάν η `msgget()` απέτυχε επειδή η ουρά μηνυμάτων υφίσταται ήδη και η σημαία `IPC_EXCL` δεν είχε οριστεί, τότε η συνάρτηση `error()` θα εκτυπώσει το ακόλουθο μήνυμα: "msgget: Resource temporarily unavailable (Ο πόρος δεν είναι προσωρινά διαθέσιμος)".

```
if (msgid < 0) {  
    perror("msgget");  
    exit(EXIT_FAILURE);  
}
```

Μετά, ορίζουμε μια συνάρτηση `f` που λαμβάνει μια παράμετρο κινητής υποδιαστολής διπλής ακρίβειας `x` και επιστρέφει μία τιμή κινητής υποδιαστολής διπλής ακρίβειας. Η συνάρτηση υπολογίζει την τιμή του $\log(x) * \sqrt{x}$ και την επιστρέφει.

```
double f(double x)  
{  
    return log(x)*sqrt(x);  
}
```

Στο επόμενο βήμα, ορίζουμε μια συνάρτηση ολοκληρώματος η οποία υπολογίζει μια κατά προσέγγιση τιμή για το καθορισμένο ολοκλήρωμα μιας συνάρτησης σε ένα καθορισμένο εύρος.

Η συνάρτηση παίρνει τρεις παραμέτρους:

α: το κάτω όριο του ολοκλήρωσης

β: το άνω όριο του ολοκλήρωσης

η: ο αριθμός των υποδιαίρεσεων, που θα χρησιμοποιηθούν στην προσέγγιση.

Η συνάρτηση υπολογίζει πρώτα το μέγεθος κάθε υποδιαίρεσης dx ως το μήκος του εύρους ολοκλήρωσης που διαιρείται με τον αριθμό των βημάτων. Στη συνέχεια, αρχικοποιεί μια μεταβλητή `S` σε 0 και εισάγει έναν βρόχο που επαναλαμβάνει `n` φορές. Σε κάθε επανάληψη του βρόχου, υπολογίζουμε την τιμή της συνάρτησης σε ένα σημείο `xi` στο τρέχον βήμα, το οποίο βρίσκεται στο μέσο του βήματος. Στη συνέχεια, η συνάρτηση προσθέτει την τιμή της συνάρτησης σε αυτό το σημείο στη μεταβλητή `S`. Αφού ολοκληρωθεί ο βρόχος, η συνάρτηση πολλαπλασιάζει το `S` επί dx για να κλιμακώσει το άθροισμα

¹⁶ **IPC_CREAT | 0666**: Σημαία η οποία στη `msgget()` να δημιουργήσει την ουρά μηνυμάτων εάν δεν υπάρχει ήδη και να ορίσει τα δικαιώματα της ουράς μηνυμάτων σε 0666. Η τιμή δικαιωμάτων 0666 καθορίζει ότι η ουρά μηνυμάτων πρέπει να είναι αναγνώσιμη και εγγράψιμη από τον κάτοχο.

¹⁷ **msgget()**: Είναι μια συνάρτηση, που ανοίγει μια υπάρχουσα ουρά μηνυμάτων ή δημιουργεί μια ουρά μηνυμάτων. Χρειάζεται δύο ορίσματα: ένα κλειδί και μια σημαία. Το κλειδί είναι μια τιμή που προσδιορίζει μοναδικά την ουρά μηνυμάτων. Δημιουργείται χρησιμοποιώντας τη συνάρτηση `ftok()` και μεταβιβάζεται στη `msgget()` ως πρώτο όρισμα.

¹⁸ **msgid**: Αποτελεί μια θετική ακέραια τιμή που χρησιμοποιείται για τον προσδιορισμό της ουράς μηνυμάτων σε επόμενες κλήσεις σε άλλες συναρτήσεις ουράς μηνυμάτων όπως `msgsnd()` και `msgrcv()`.

¹⁹ **perror()**: Εκτυπώνει το μήνυμα σφάλματος που σχετίζεται με την πιο πρόσφατη κλήση σε μία λειτουργία συστήματος.

ανάλογα με το μέγεθος κάθε βήματος και στη συνέχεια επιστρέφει το αποτέλεσμα. Συνεπώς, χρησιμοποιούμε την συνάρτηση με στόχο να προσεγγίσουμε το ολοκλήρωμα μίας συνάρτησης σε ένα καθορισμένο εύρος, καλώντας την με την επιθυμητή συνάρτηση, εύρος και αριθμό βημάτων.

Π.χ.

```
double result = integrate(0.0, 1.0, 1000);
```

Υπολογίζει μια κατά προσέγγιση τιμή για το ολοκλήρωμα της συνάρτησης f στην περιοχή από 0,0 έως 1,0, χρησιμοποιώντας 1000 βήματα. Το αποτέλεσμα θα εκχωρηθεί στη μεταβλητή διπλής ακρίβειας `result`.

```
double integrate(double a, double b, unsigned long n)
{
    double dx = (b - a) / n;
    double S = 0;
    for (unsigned long i = 0; i < n; i++) {
        double xi = a + (i + 0.5) * dx;
        S += f(xi);
    }
    S *= dx;
    return S;
}
```

Στο επόμενο βήμα, ορίζουμε τη συνάρτηση `get_wtime()` στόχος της οποίας είναι να επιστρέφει την τρέχουσα ώρα σε `sec`, με ακρίβεια `msec`. Αυτό επιτυγχάνεται με τη συνάρτηση `gettimeofday`²⁰. Πιο αναλυτικά, καλείται πρώτα η `gettimeofday` και μεταδίδει έναν δείκτη σε έναν δείκτη χρονικής τιμής `struct` και σε έναν δείκτη `NULL`. Αυτό αναθέτει την τιμή της τρέχουσας ώρας στο πεδίο `timeval` της δομής. Στη συνέχεια, η συνάρτηση επιστρέφει το άθροισμα των πεδίων `tv_sec` και `tv_usec`, που έχουν μετατραπεί σε τύπο `double`. Το πεδίο `tv_usec` πολλαπλασιάζεται επί $1,0e-6$ ($0,000001$) για να μετατραπεί από `msec` σε `sec`. Το τελικό αποτέλεσμα είναι ο τρέχων χρόνος σε `sec`, με ακρίβεια `msec`. Όμως ο χρόνος που έχει παρέλθει δεν θα είναι ακριβής εάν ο κώδικας εκτελείται για μεγάλο χρονικό διάστημα, καθώς η συνάρτηση `gettimeofday` μπορεί να επιστρέψει την ίδια τιμή πολλές φορές εάν η ώρα δεν αλλάζει μεταξύ των κλήσεων. Ίσως, για να λάβουμε έναν πιο ακριβή χρόνο που έχει παρέλθει, θα μπορούσαμε να χρησιμοποιήσουμε ένα χρονόμετρο υψηλότερης ανάλυσης, όπως τη συνάρτηση `clock_gettime`.

```
double get_wtime(void)
{
    struct timeval t;
    gettimeofday(&t, NULL);
    return (double)t.tv_sec + (double)t.tv_usec*1.0e-6;
}
```

²⁰ **gettimeofday()**: Είναι μια κλήση συστήματος που επιστρέφει την τρέχουσα ώρα.

παίρνει ως όρισμα ένα αντικείμενο `timeval` δομής, το οποίο χρησιμοποιείται για την αποθήκευση της τρέχουσας ώρας. Το πεδίο `tv_sec` αυτής της δομής κρατά τον χρόνο σε `seconds` και το πεδίο `tv_usec` τον χρόνο σε `mseconds`.

Το επόμενο βήμα αποτελείται από έναν βρόχο που επαναλαμβάνεται `num_processes` φορές για την δημιουργία `num_processes`, σε πλήθος, διεργασιών. Μέσα στον βρόχο, υπάρχει μια δήλωση `if` που καλεί τη συνάρτηση `fork()`²¹. Η νέα διαδικασία ονομάζεται θυγατρική διαδικασία(CHILD) και η διαδικασία κλήσης ονομάζεται γονική διαδικασία(PARENT). Εάν η συνάρτηση `fork()` είναι επιτυχής, επιστρέφει μια τιμή 0 στη θυγατρική διεργασία και το αναγνωριστικό διεργασίας της θυγατρικής διαδικασίας στη γονική διεργασία. Εάν η συνάρτηση `fork()` αποτύχει, επιστρέφει -1 στη διαδικασία κλήσης.

Η εντολή `if (fork() == 0) { ... }` ελέγχει εάν η τιμή που επιστρέφεται από την `fork()` είναι 0, πράγμα που δείχνει ότι η τρέχουσα διεργασία είναι η θυγατρική διεργασία. Εάν η τρέχουσα διεργασία είναι η θυγατρική, τότε εκτελείται ο κώδικας μέσα στο μπλοκ `if`. Μέσα στο μπλοκ `if`, η θυγατρική διαδικασία υπολογίζει το ολοκλήρωμα και αποθηκεύει το αποτέλεσμα σε μια μεταβλητή που ονομάζεται αποτέλεσμα. Στη συνέχεια, δημιουργείται ένα μήνυμα χρησιμοποιώντας μια δομή που ονομάζεται `struct message`, η οποία περιέχει τον τύπο του μηνύματος (`i+1`) και το αποτέλεσμα του ολοκληρώματος (αποτέλεσμα). Στη συνέχεια καλείται η συνάρτηση `msgsnd()` για να στείλει το μήνυμα στη γονική διεργασία. Η συνάρτηση `msgsnd()` παίρνει τέσσερα ορίσματα: το αναγνωριστικό της ουράς μηνυμάτων, έναν δείκτη στο μήνυμα, το μέγεθος του

```
// Create child processes
for (int i = 0; i < num_processes; i++) {
    if (fork() == 0) {
        // Child process: compute integral and send result to parent
        double a = 1.0;
        double b = 4.0;
        unsigned long n = 1e9;
        double result = integrate(a, b, n); // compute integral
        struct message msg = { i+1, result };
        msgsnd(msgid, &msg, sizeof(msg), 0);
        exit(EXIT_SUCCESS);
    }
}
```

Το επόμενο βήμα στον κώδικα αρχικοποιεί την μεταβλητή `total` σε 0 και στη συνέχεια εισάγει έναν βρόχο ο οποίος επαναλαμβάνει φορές `num_processes`. Σε κάθε επανάληψη του βρόχου, ο κώδικας χρησιμοποιεί τη συνάρτηση `msgrcv` με σκοπό να λάβει ένα μήνυμα από την ουρά μηνυμάτων, το οποίο προσδιορίζεται από την παράμετρο `msgid`. Το μήνυμα αποθηκεύεται στο στιγμιότυπο `msg` του `struct message` το οποίο περιέχει ένα πεδίο που ονομάζεται `result` και περιέχει το αποτέλεσμα ορισμένων υπολογισμών που εκτελούνται από μια θυγατρική διεργασία(διεργασία παιδί). Έπειτα, υπολογίζουμε τον χρόνο που έχει παρέλθει λαμβάνοντας τη διαφορά μεταξύ της τρέχουσας ώρας (`t1`) και της ώρας έναρξης (`t0`). Η συνάρτηση `get_wtime` χρησιμοποιείται για τη λήψη της τρέχουσας ώρας σε δευτερόλεπτα, με ακρίβεια μικροδευτερόλεπτων. Ο χρόνος που έχει παρέλθει υπολογίζεται λαμβάνοντας τη διαφορά μεταξύ `t1` και `t0` και αποθηκεύοντάς την στη μεταβλητή `t1 - t0`. Αυτή η τιμή στη συνέχεια εκτυπώνεται χρησιμοποιώντας τη συνάρτηση `printf`.

Ο κώδικας προσθέτει την τιμή `msg.result` στη μεταβλητή `total` και μετά συνεχίζει στην επόμενη επανάληψη του βρόχου. Μετά την ολοκλήρωση του βρόχου, ο κώδικας εκτυπώνει την τιμή του αποτελέσματος χρησιμοποιώντας τη συνάρτηση `printf`.

²¹ **fork()**: Η συνάρτηση `fork()` δημιουργεί μια νέα διαδικασία αντιγράφοντας τη διαδικασία κλήσης

```
// Parent process: receive results from child processes and add them up
double total = 0;

double t0 = get_wtime();
for (int i = 0; i < num_processes; i++) {

    struct message msg;
    msgrcv(msgid, &msg, sizeof(msg), 0, 0);

    total += msg.result;

    double t1 = get_wtime();
printf("Time in seconds: %lf\n ", t1-t0);

}

printf("Total result: %f\n", total);
```

Τέλος, στόχος μας είναι να καθαρίζουμε τις ουρές μηνυμάτων όταν δεν χρειάζονται πλέον, για να τις αποτρέψουμε από το να καταναλώνουν άσκοπα πόρους του συστήματος. Έτσι, χρησιμοποιούμε τη συνάρτηση `msgctl` για να αναγνωρίσουμε την ουρά μηνυμάτων κατά την αποστολή και λήψη μηνυμάτων και έπειτα να την αφαιρέσουμε(την ουρά) από το σύστημα. Με την βοήθεια της σταθεράς `IPC_RMID` αφαιρούμε την ουρά μηνυμάτων και η παράμετρος `NULL` είναι ένα σύμβολο κράτησης θέσης για πρόσθετα ορίσματα που δεν χρησιμοποιούνται σε αυτήν την περίπτωση.

```
// Delete message queue
msgctl(msgid, IPC_RMID, NULL);
```

```
return 0;
}
```

Η επιστροφή της τιμής 0, υποδεικνύει ότι το πρόγραμμα ολοκληρώθηκε με επιτυχία και επιστρέφει μια τιμή 0 στο λειτουργικό σύστημα.

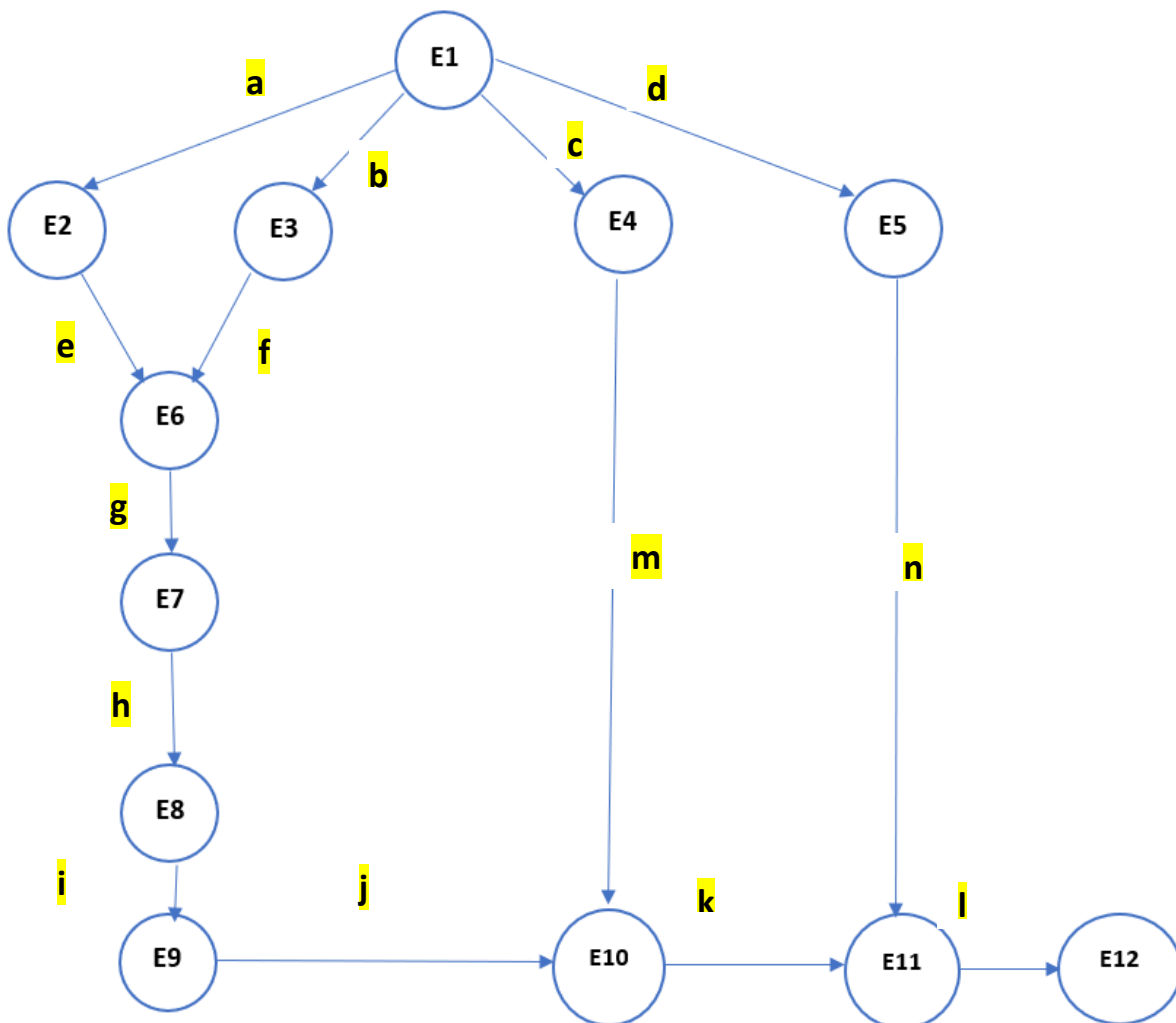
3^η ΑΣΚΗΣΗ:

ερώτημα α:

Σύμφωνα με την εκφώνηση οι εργασίες στο εργοστάσιο πραγματοποιούνται ως εξής:

- Πρώτα εκτελείται η E1,
- Μετά εκτελούνται παράλληλα οι E2, E3, E4, E5
- Αφού εκτελεστούν οι E2 και E3 εκτελούνται στη σειρά οι E6, E7, E8, E9
- Η E10 εκτελείται μετά την E9 και E4
- Η E11 εκτελείται μετά την E10 και την E5
- Τέλος η E12 εκτελείται μετά την E11

Προκύπτει λοιπόν το παρακάτω διάγραμμα προτεραιότητας στο οποίο αναγράφονται και οι σημαφόροι που χρειάζονται :



ερώτημα β:

repeat // Οι διεργασίες εκτελούνται επαναληπτικά

```
begin
    E1;
cobegin
    begin
        cobegin
            begin
                cobegin
                    E2;
                    E3;
                coend;
            E6;
            E7;
            E8;
            E9;
        end;
    E4;
    coend;
    E10;
end;
E5;
coend;
E11;
E12;
end;
forever;
```

ερώτημα γ:

Για να διασφαλισθεί η σωστή σειρά εκτέλεσης μπορούν αρχικά να εφαρμοστούν τόσοι δυαδικοί σημαφόροι (mutex) όσες και οι εξαρτήσεις μεταξύ των διεργασιών. Όπως φαίνονται παραπάνω στο σχήμα χρησιμοποιούνται 14 mutex με αρχική τιμή 0. Επίσης για να μπορεί η διεργασία E12 να συσκευάζει ανα δωδεκάδες έχει οριστεί μια μεταβλητή bottles για την καταμέτρηση των έτοιμων μπουκαλιών και ένας δυαδικός σημαφόρος packages που ενεργοποιείται κάθε φορά που υπάρχουν 12 διαθέσιμα μπουκάλια.

```

repeat
semaphore a,b,c,d,e,f,g,h,i,j,kl,m,n,packages;

a=b=c=d=e=f=g=h=i=j=k=l=m=n=packages = 0;

shared int bottles = 0;      // ο συνολικός αριθμός μπουκαλιών που έχουν παραχθεί

cobegin

    begin E1; signal(a); signal(b); signal(c); signal(d); end;
    begin wait(a); E2; signal(e); end;
    begin wait(b); E3; signal(f); end;
    begin wait(c); E4; signal(m); end;
    begin wait(d); E5; signal(n); end;
    begin wait(e); wait(f); E6; signal(g); end;
    begin wait(g); E7; signal(h); end;
    begin wait(h); E8; signal(i); end;
    begin wait(i); E9; signal(j); end;
    begin
        wait(j);
        wait(m);

        E10;

        bottles = bottles + 1; // Μετά την εκτέλεση της E10 καταμετρούνται τα  μπουκάλια
        // ο σημαφόρος packages ενεργοποιείται όταν συμπληρώνεται μια δωδεκάδα
        if(bottles % 12 == 0) signal(packages);
        signal(k);
    end;
    begin wait(k); wait(n); E11; signal(l); end;
    begin
        wait(l);

        wait(packages); //όσο δεν υπάρχουν δώδεκα μπουκάλια η διεργασία μένει αδρανής
        E12;
    end;

coend;

forever;

```

ερώτημα δ:

Το πρόβλημα μπορεί να απλοποιηθεί με την χρήση 8 δυαδικών σηματοφόρων για τον συγχρονισμό των διεργασιών.
Αναλυτικότερα:

- Οι σηματοφόροι a, b, c, d μπορούν να αντικατασταθούν από ένα σηματοφόρο - A- καθώς εκτελούνται μετά την E1.
- Οι σηματοφόροι e, f μπορούν να αντικατασταθούν με ένα σηματοφόρο B ο οποίος είναι αρχικοποιημένος με -1.
- Αντίστοιχα τα ζεύγη σηματοφόρων j,m και k,n αντικαθίστανται από ένα σηματοφόρο το καθένα με αρχική τιμή -1.

```
repeat
semaphore A,B,C,D,E,F,G,packages;
A=C=D=E=H=packages = 0;
B=F=G= -1
shared int bottles = 0;      // ο συνολικός αριθμός μπουκαλιών που έχουν παραχθεί

cobegin
    begin E1; signal(A); end;
    begin wait(A); E2; signal(B); end;
    begin wait(A); E3; signal(B); end;
    begin wait(A); E4; signal(F); end;
    begin wait(A); E5; signal(G); end;
    begin wait(B); E6; signal(C); end;
    begin wait(C); E7; signal(D); end;
    begin wait(D); E8; signal(E); end;
    begin wait(E); E9; signal(F); end;
    begin
        wait(F);
        E10;
        bottles = bottles + 1; // Μετά την εκτέλεση της E10 καταμετρούνται τα μπουκάλια
        // ο σηματοφόρος packages ενεργοποιείται όταν συμπληρώνεται μια δωδεκάδα
        if(bottles % 12 == 0) signal(packages);
        signal(G);
    end;
    begin wait(G); E11; signal(H); end;
    begin
        wait(H);
        wait(packages); //όσο δεν υπάρχουν δώδεκα μπουκάλια η διεργασία μένει αδρανής
        E12;
    end;
coend;
forever;
```

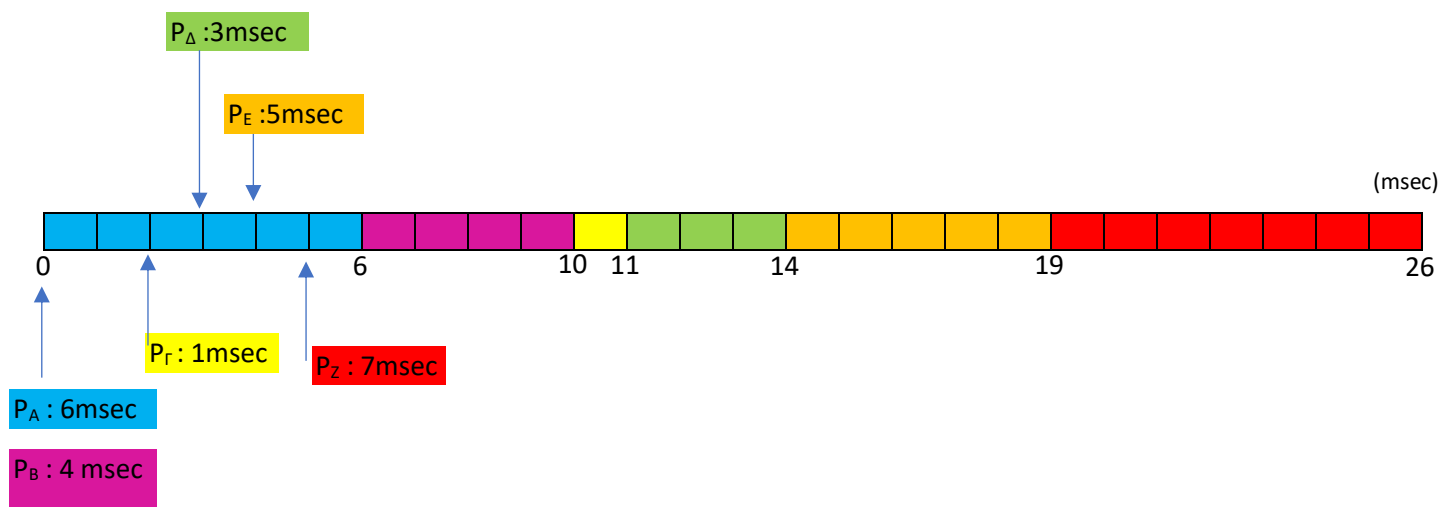
4^η ΆΣΚΗΣΗ: Χρονοπρογραμματισμός Διεργασιών

Διεργασία	Χρόνος άφιξης (XA)	Διάρκεια εκτέλεσης (XE)	PID
A	0	6	3
B	0	4	1
Γ	2	1	2
Δ	3	3	5
E	4	5	4
Z	5	7	1

- First – come first – served algorithm (FCFS)**

Ο αλγόριθμος **FCFS** είναι ένας μη προεκτοπιστικός ²² αλγόριθμος για συστήματα δέσμης, ο οποίος καθορίζει ότι η CPU θα εκτελεί κάθε διεργασία με βάση την σειρά άφιξης της στην ουρά με τις διεργασίες οι οποίες είναι έτοιμες προς εκτέλεση και αιτούνται την χρήση της CPU.

✓ Διάγραμμα Gantt:



✓ Χρόνοι Διεκπεραίωσης²³:

$$X_{\Delta A} = 6 - 0 = 6 \text{ msec} \quad X_{\Delta \Delta} = 14 - 3 = 11 \text{ msec}$$

$$X_{\Delta B} = 10 - 0 = 10 \text{ msec} \quad X_{\Delta E} = 19 - 4 = 15 \text{ msec}$$

$$X_{\Delta \Gamma} = 11 - 2 = 9 \text{ msec} \quad X_{\Delta Z} = 26 - 5 = 21 \text{ msec}$$

²² **μη προεκτοπιστικοί** (nonpreemptive) αλγόριθμοι χρονοπρογραμματισμού είναι αυτοί που επιλέγουν μια διεργασία και της επιτρέπουν να εκτελείται μέχρι να μπλοκαριστεί (από I/O ή επειδή αναμένει κάποια άλλη διεργασία) ή μέχρι να επιστέψει εθελοντικά τον έλεγχο της CPU. Σε αυτή την κατηγορία αλγορίθμων, ακόμα και αν η διεργασία εκτελείτε επί ώρες, κανένας δεν θα την σταματήσει από το να έχει τον έλεγχο της CPU, το σύστημα θα περιμένει να τελειώσει η εκτέλεση της.

²³ **Χρόνος διεκπεραίωσης** είναι ο χρόνος που μεσολαβεί από την άφιξη της διεργασίας στην ουρά μέχρι και το τέλος της εκτέλεσής της. Είναι ίδιος με τον χρόνο ολοκλήρωσης της διεργασίας.

✓ Χρόνοι αναμονής²⁴ :

$$XA_A = 0 \text{ msec}$$

$$XA_\Delta = 11 - 3 = 8 \text{ msec}$$

$$XA_B = 6 - 0 = 6 \text{ msec}$$

$$XA_E = 14 - 4 = 10 \text{ msec}$$

$$XA_\Gamma = 10 - 2 = 8 \text{ msec}$$

$$XA_Z = 19 - 5 = 14 \text{ msec}$$

Άρα :

$$\checkmark \text{ Μέσος Χρόνος Αναμονής} = \frac{XA_A + XA_B + XA_\Gamma + XA_\Delta + XA_E + XA_Z}{6} = \frac{0 + 6 + 8 + 8 + 10 + 14}{6} \approx 7.666 \text{ msec}$$

$$\checkmark \text{ Μέσος Χρόνος Ολοκλήρωσης} = \frac{XD_A + XD_B + XD_\Gamma + XD_\Delta + XD_E + XD_Z}{6} = \frac{6 + 10 + 9 + 11 + 15 + 21}{6} = 12 \text{ msec}$$

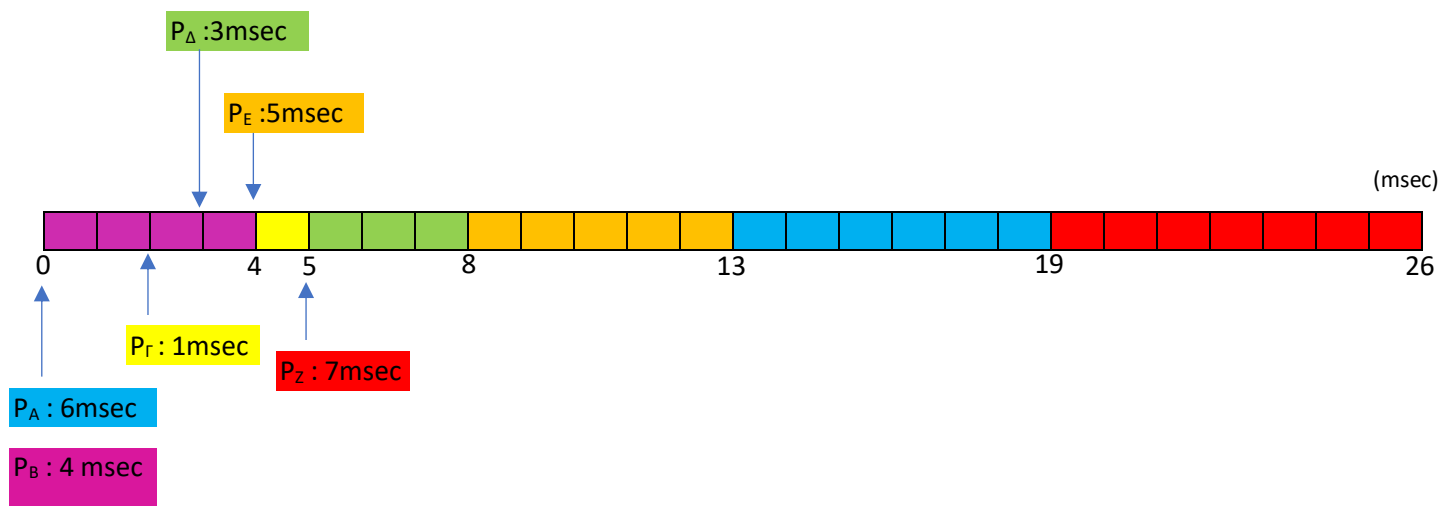
$$\checkmark \text{ Μέσος Χρόνος Απόκρισης}^{25} = \frac{0 + 6 + 8 + 8 + 10 + 14}{6} \approx 7.666 \text{ msec}$$

✓ Πλήθος αλλαγών διεργασίας που εκτελείται στον επεξεργαστή (context switch): 6

- **Shortest Job First algorithm (SJF)**

Ο αλγόριθμος **SJF χωρίς προεκτόπιση** είναι ένας αλγόριθμος για συστήματα δέσμης, κατά τον οποίο οι διεργασίες εκτελούνται μέχρι να ολοκληρωθούν σύμφωνα με τη διάρκεια τους και συγκεκριμένα από τη διεργασία με τη μικρότερη διάρκεια προς τη διεργασία με τη μεγαλύτερη διάρκεια. Αν έχουμε διεργασίες με ίδια διάρκεια τότε αυτές εκτελούνται με βάση τη σειρά άφιξης τους στη ουρά. Στον **SJF** υποτίθεται γνωρίζονται προκαταβολικά οι χρόνοι που χρειάζονται για να ολοκληρωθούν οι διεργασίες.

✓ Διάγραμμα Gantt:



✓ Χρόνοι Διεκπεραίωσης:

$$XD_A = 19 - 0 = 19 \text{ msec}$$

$$XD_\Delta = 8 - 3 = 5 \text{ msec}$$

$$XD_B = 4 - 0 = 4 \text{ msec}$$

$$XD_E = 13 - 4 = 9 \text{ msec}$$

$$XD_\Gamma = 5 - 2 = 3 \text{ msec}$$

$$XD_Z = 26 - 5 = 21 \text{ msec}$$

²⁴ Χρόνος αναμονής : χρόνος που η διεργασία βρίσκεται σε κατάσταση αναμονής.

²⁵ Χρόνος Απόκρισης: χρόνος από την υποβολή ενός αιτήματος μέχρι να παραχθεί η πρώτη απόκριση

✓ Χρόνοι αναμονής :

$$XA_A = 13 \text{ msec}$$

$$XA_\Delta = 5 - 3 = 2 \text{ msec}$$

$$XA_B = 0 \text{ msec}$$

$$XA_E = 8 - 4 = 4 \text{ msec}$$

$$XA_\Gamma = 4 - 2 = 2 \text{ msec}$$

$$XA_Z = 19 - 5 = 14 \text{ msec}$$

Άρα :

$$\checkmark \text{ Μέσος Χρόνος Αναμονής} = \frac{XA_A + XA_B + XA_\Gamma + XA_\Delta + XA_E + XA_Z}{6} = \frac{13 + 0 + 2 + 2 + 4 + 14}{6} \approx 5.83333 \text{ msec}$$

$$\checkmark \text{ Μέσος Χρόνος Ολοκλήρωσης} = \frac{X\Delta_A + X\Delta_B + X\Delta_\Gamma + X\Delta_\Delta + X\Delta_E + X\Delta_Z}{6} = \frac{19 + 4 + 3 + 5 + 9 + 21}{6} \approx 10.166 \text{ msec}$$

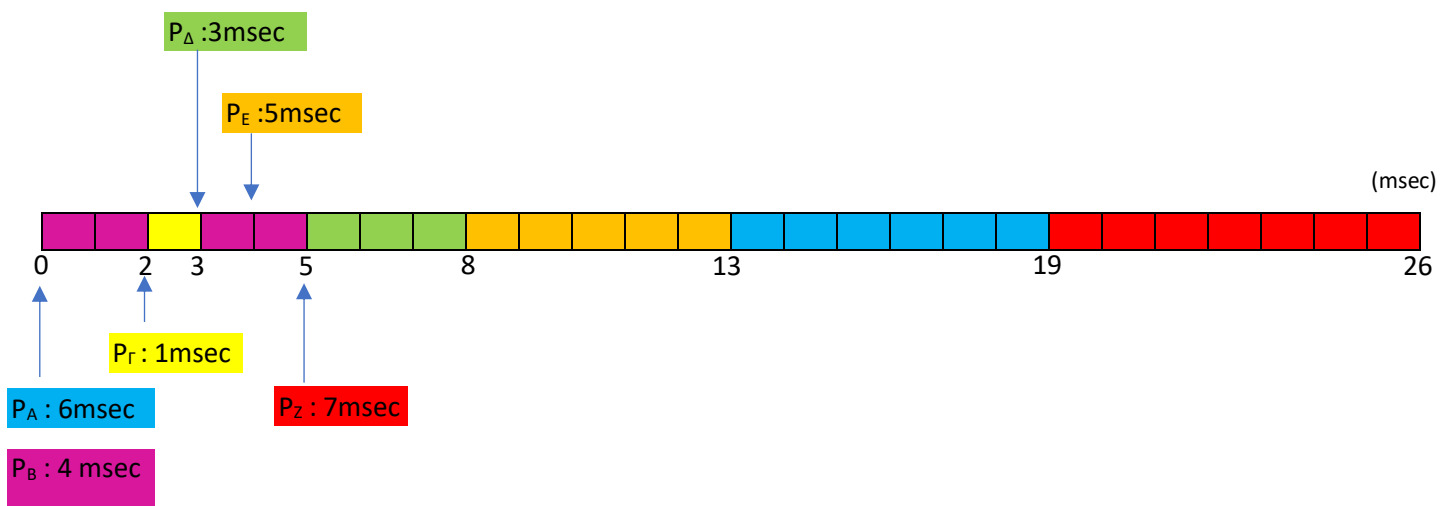
$$\checkmark \text{ Μέσος Χρόνος Απόκρισης} = \frac{0 + 2 + 2 + 4 + 13 + 14}{6} \approx 5.83333 \text{ msec}$$

✓ Πλήθος αλλαγών διεργασίας που εκτελείται στον επεξεργαστή ((context switch): 6

- **Shortest Remaining Time First algorithm (SRTF)**

Ο αλγόριθμος **SRTF** είναι μια προεκτοπιστική²⁶ εκδοχή του **SJF** κατά τον οποίο επιλέγεται η διεργασία με τον μικρότερο εναπομείναντα χρόνο. Με την άφιξη κάθε νέας διεργασίας η οποία αιτείται να εκτελεστεί γίνεται σύγκριση μεταξύ του χρόνου εκτέλεσης της νέας διεργασίας με τον απομένοντα χρόνο της τρέχουσας διεργασίας. Αν η διεργασία χρειάζεται λιγότερο χρόνο εκτέλεσης, τότε η τρέχουσα διεργασία αναστέλλεται και η νέα εργασία ξεκινά να εκτελείται. Και πάλι υποτίθεται γνωρίζονται προκαταβολικά οι χρόνοι που χρειάζονται για να ολοκληρωθούν οι διεργασίες.

✓ Διάγραμμα Gantt:



²⁶ **προεκτοπιστικοί** (preemptive) αλγόριθμοι χρονοπρογραμματισμού είναι αυτοί που επιλέγουν μια διεργασία και της επιτρέπουν να εκτελεστεί για ένα καθορισμένο μέγιστο χρονικό διάστημα. Αν η διεργασία συνεχίζει να εκτελείται μετά την λήξη του συγκεκριμένου χρονικού διαστήματος, η διεργασία αναστέλλεται και επιλέγεται κάποια άλλη προς εκτέλεση.

✓ Χρόνοι Διεκπεραίωσης:

$$\begin{aligned} X\Delta_A &= 19 - 0 = 19\text{msec} & X\Delta_\Delta &= 8 - 3 = 5\text{ msec} \\ X\Delta_B &= 5 - 0 = 5\text{msec} & X\Delta_E &= 13 - 4 = 9\text{ msec} \\ X\Delta_\Gamma &= 3 - 2 = 1\text{msec} & X\Delta_Z &= 26 - 5 = 21\text{ msec} \end{aligned}$$

✓ Χρόνοι αναμονής :

$$\begin{aligned} XA_A &= 13 - 0 = 13\text{ msec} & XA_\Delta &= 5 - 3 = 2\text{ msec} \\ XA_B &= 3 - 2 = 1\text{ msec} & XA_E &= 8 - 4 = 4\text{ msec} \\ XA_\Gamma &= 2 - 2 = 0\text{ msec} & XA_Z &= 19 - 5 = 14\text{ msec} \end{aligned}$$

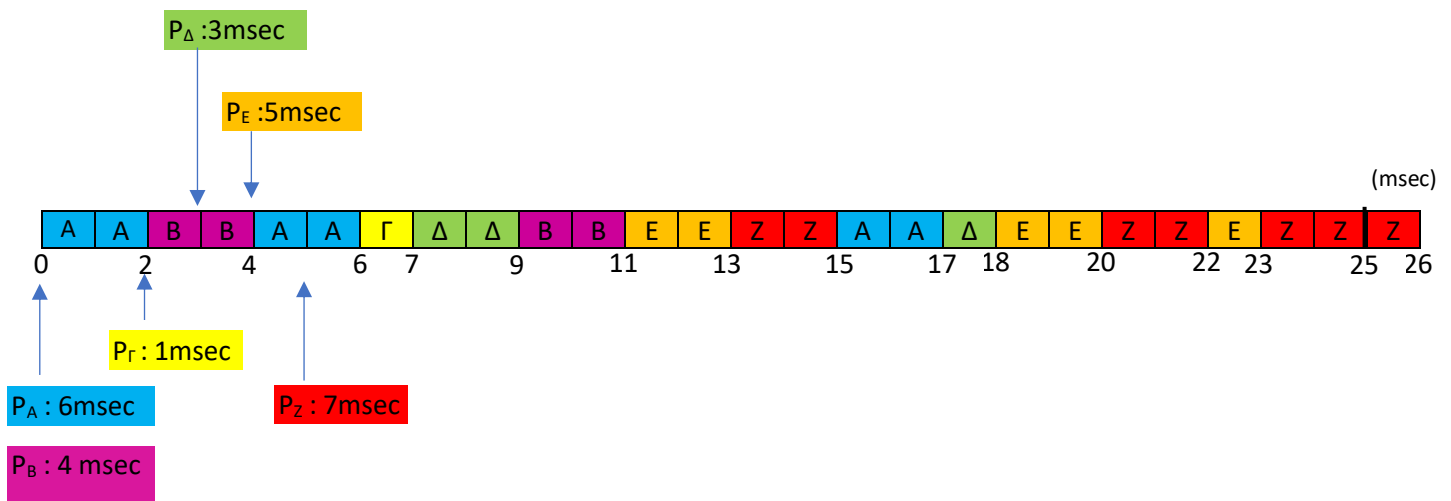
Άρα :

- ✓ Μέσος Χρόνος Αναμονής $= \frac{XA_A + XA_B + XA_\Gamma + XA_\Delta + XA_E + XA_Z}{6} = \frac{13 + 1 + 0 + 2 + 4 + 14}{6} \approx 5.666\text{ msec}$
- ✓ Μέσος Χρόνος Ολοκλήρωσης $= \frac{X\Delta_A + X\Delta_B + X\Delta_\Gamma + X\Delta_\Delta + X\Delta_E + X\Delta_Z}{6} = \frac{19 + 5 + 1 + 5 + 9 + 21}{6} = 10\text{msec}$
- ✓ Μέσος Χρόνος Απόκρισης $= \frac{13 + 0 + 0 + 2 + 4 + 14}{6} = 5.5\text{ msec}$
- ✓ Πλήθος αλλαγών διεργασίας που εκτελείται στον επεξεργαστή (context switch): 7

- **Round Robin algorithm(RR) with quantum time =2msec**

Ο αλγόριθμος RR είναι ένας προεκτοπιστικός αλγόριθμος με εφαρμογή στα αλληλεπιδραστικά συστήματα, κατά τον οποίο κάθε διεργασία εκτελείτε για ένα συγκεκριμένο χρονικό διάστημα το οποίο ονομάζεται κβάντο χρόνου (quantum time) , με βάση την σειρά άφιξης της στην ουρά με τις διεργασίες οι οποίες είναι έτοιμες προς εκτέλεση και αιτούνται την χρήση της CPU. Κάθε εκτέλεση διαρκεί όσο και το κβάντο .Αν δεν ολοκληρωθεί η εκτέλεση της διεργασίας εντός του κβάντου, διακόπτεται και μπαίνει στην ουρά αναμονής και εκτελείται πάλι μέχρι να τελειώσει. Αν τελειώσει νωρίτερα από το χρόνο που ορίζει το κβάντο, τότε εγκαταλείπει τη CPU χωρίς να έχει ολοκληρωθεί το κβάντο. Ο χρονοπρογραμματιστής επιλέγει στη συνέχεια την επόμενη διεργασία που περιμένει στην ουρά αναμονής και την εκτελεί.

✓ Διάγραμμα Gantt:



Ουρά αναμονής

Χρονική Στιγμή(msec)	0	2	2	3	4	4	5	6	9	13	15	20	22	25
Διεργασία	B	A ΥΠΟΛΟΙΠΟ: 4msec	Γ	Δ	B ΥΠΟΛΟΙΠΟ: 2msec	E	Z	A ΥΠΟΛΟΙΠΟ: 2msec	Δ ΥΠΟΛΟΙΠΟ: 1msec	E ΥΠΟΛΟΙΠΟ: 3msec	Z ΥΠΟΛΟΙΠΟ: 5msec	E ΥΠΟΛΟΙΠΟ: 1msec	Z ΥΠΟΛΟΙΠΟ: 3msec	Z ΥΠΟΛΟΙΠΟ: 1msec

✓ Χρόνοι Διεκπεραίωσης:

$$X_{\Delta A} = 17 - 0 = 17 \text{msec} \quad X_{\Delta \Delta} = 18 - 3 = 15 \text{msec}$$

$$X_{\Delta B} = 11 - 0 = 11 \text{msec} \quad X_{\Delta E} = 23 - 4 = 19 \text{msec}$$

$$X_{\Delta \Gamma} = 7 - 2 = 5 \text{msec} \quad X_{\Delta Z} = 26 - 5 = 21 \text{msec}$$

✓ Χρόνοι αναμονής :

$$X_{A_A} = (4 - 2) + (15 - 6) = 2 + 9 = 11 \text{msec} \quad X_{A_{\Delta}} = (7 - 3) + (17 - 9) = 4 + 8 = 12 \text{msec}$$

$$X_{A_B} = (2 - 0) + (9 - 4) = 2 + 5 = 7 \text{msec} \quad X_{A_E} = (11 - 4) + (18 - 13) + (22 - 20) = 7 + 5 + 2 = 14 \text{msec}$$

$$X_{A_{\Gamma}} = 6 - 2 = 4 \text{msec} \quad X_{A_Z} = (13 - 5) + (20 - 15) + (23 - 22) + (25 - 25) = 14 \text{msec}$$

Άρα :

$$✓ \text{ Μέσος Χρόνος Αναμονής} = \frac{X_{A_A} + X_{A_B} + X_{A_{\Gamma}} + X_{A_{\Delta}} + X_{A_E} + X_{A_Z}}{6} = \frac{11 + 7 + 4 + 12 + 14 + 14}{6} \approx 10.333 \text{msec}$$

$$✓ \text{ Μέσος Χρόνος Ολοκλήρωσης} = \frac{X_{\Delta A} + X_{\Delta B} + X_{\Delta \Gamma} + X_{\Delta \Delta} + X_{\Delta E} + X_{\Delta Z}}{6} = \frac{17 + 11 + 5 + 15 + 19 + 21}{6} \approx 14.666 \text{msec}$$

$$✓ \text{ Μέσος Χρόνος Απόκρισης} = \frac{0 + 2 + 4 + 4 + 7 + 8}{6} \approx 4.1666 \text{msec}$$

✓ Πλήθος αλλαγών διεργασίας που εκτελείται στον επεξεργαστή (context switch): 15

Ποσοστά μεταβολής Μέσου Χρόνου Διεκπεραίωσης σε σχέση με τον FCFS

$$1. \text{ SJF: } \text{Ποσοστό μεταβολής} = \frac{MX\Delta_{SJF} - MX\Delta_{FCFS}}{MX\Delta_{FCFS}} * 100\% = \frac{10.166 - 12}{12} * 100\% \approx -15.2833\%$$

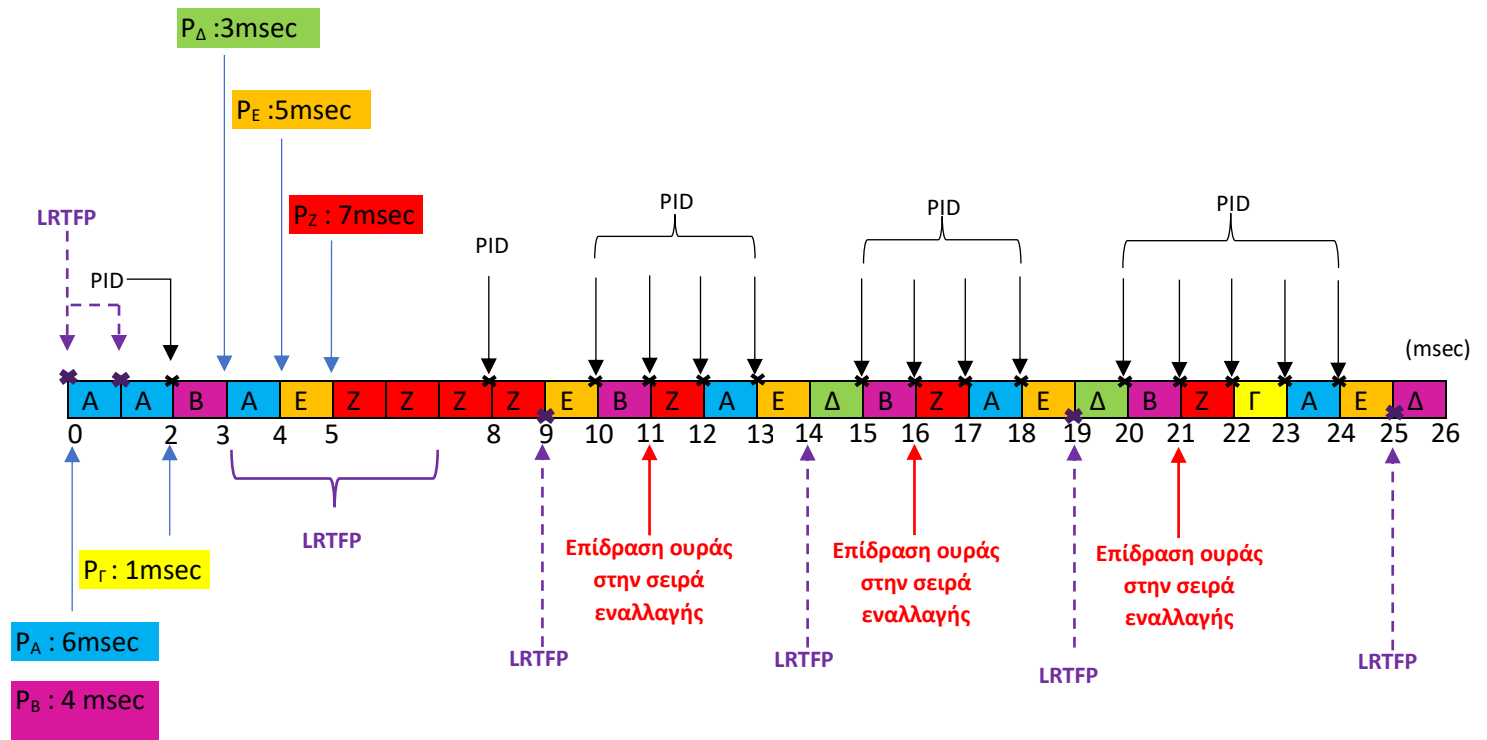
$$2. \text{ SRTF: } \text{Ποσοστό μεταβολής} = \frac{MX\Delta_{SRTF} - MX\Delta_{FCFS}}{MX\Delta_{FCFS}} * 100\% = \frac{10 - 12}{12} * 100\% \approx -16.6666\%$$

$$3. \text{ RR: } \text{Ποσοστό μεταβολής} = \frac{MX\Delta_{RR} - MX\Delta_{FCFS}}{MX\Delta_{FCFS}} * 100\% = \frac{14.666 - 12}{12} * 100\% \approx 22.2166\%$$

- **Longest Remaining Time First Preemptive algorithm (LRTFP)**

Ο αλγόριθμος **LRTFP** είναι ένας προεκτοπιστικός αλγόριθμος χρονοδρομολόγησης διεργασιών ο οποίος εκτελεί πρώτα τις διεργασίες με τον μεγαλύτερο απομένοντα χρόνο, σε κάθε χρονική στιγμή άφιξης ή εκτέλεσης. Ο αλγόριθμος εξετάζει πρώτα την ιδιότητα LRTF για την προεκτόπιση και σε περιπτώσεις ισοδυναμίας ενεργοποιείται η PID προτεραιότητα, δίνοντας προτεραιότητα στη διαδικασία με το μικρότερο αναγνωριστικό προτεραιότητας διεργασίας (PID- Priority ID) και σύμφωνα με τις ουρές αφίξεων και εναλλαγών. Και πάλι υποτίθεται γνωρίζονται προκαταβολικά οι χρόνοι που χρειάζονται για να ολοκληρωθούν οι διεργασίες.

✓ Διάγραμμα Gantt:



Ουρά Αναμονής

Χρονική Στιγμή	0	2	2	3	3	4	5	9	10	11	12	13	14	15	16	17	18	19	20
Διεργασίες	B rest: 4msec	A rest: 4msec	Γ rest: 1msec	B rest: 3msec	Δ rest: 3msec	A rest: 3msec	E rest: 4msec	Z rest: 3msec	E rest: 3msec	B rest: 2msec	Z rest: 2msec	A rest: 2msec	E rest: 2msec	Δ rest: 4msec	B rest: 1msec	Z rest: 1msec	A rest: 1msec	E rest: 1msec	Δ rest: 1msec

LTRFP explanation :

1) $t = 0 \text{ msec}$

Έχουν αφιχθεί μόνο οι διεργασίες A και B με χρόνο εκτέλεσης_A = 6msec και χρόνο εκτέλεσης_B = 4msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time A > remaining time B

➔ Άρα αρχίζει η εκτέλεση της διεργασίας A μέχρι την $t = 2 \text{ sec}$ που οι απομείναντες χρόνοι των 2 διεργασιών εξισώνονται.

2) $t = 3 \text{ msec}$

Ισχύει ότι remaining time B είναι 3msec ενώ για την διεργασία A η οποία βρίσκεται στην ουρά αναμονής remaining time A είναι 4 msec. Ταυτόχρονα φτάνει στο σύστημα η διεργασία Δ με χρόνο εκτέλεσης ίσο με 3 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time A > remaining time B.

➔ Αναστέλλεται η διεργασία B και να πάει στην ουρά αναμονής με την διεργασία Δ

➔ Συνεχίζεται η εκτέλεση της διεργασίας A για άλλο 1 msec.

3) $t = 4 \text{ msec}$

Ισχύει ότι remaining time A = remaining time B = remaining time Δ = 3 msec και ταυτόχρονα φτάνει στο σύστημα η διεργασία E με χρόνο εκτέλεσης ίσο με 5 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time E > remaining time A.

➔ Αναστέλλεται η διεργασία A και να πάει στην ουρά αναμονής με τις διεργασίες Δ και B.

➔ Αρχίζει η εκτέλεση της διεργασίας E αφού έχει τον μεγαλύτερο απομένοντα χρόνο εκτέλεσης.

4) $t = 5 \text{ msec}$

Ισχύει ότι remaining time E = 4 msec και remaining time A = remaining time B = remaining time Δ = 3 msec. Ταυτόχρονα φτάνει στο σύστημα η διεργασία Z με χρόνο εκτέλεσης ίσο με 7 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time Z > remaining time E.

➔ Αναστέλλεται η διεργασία E και να πάει στην ουρά αναμονής με τις διεργασίες Δ, A και B.

➔ Αρχίζει η εκτέλεση της διεργασίας Z για τα επόμενα 3 msec αφού έχει τον μεγαλύτερο απομένοντα χρόνο εκτέλεσης.

5) $t = 9 \text{ msec}$

Ισχύει ότι remaining time E = 4 msec και remaining time Z = remaining time A = remaining time B = remaining time Δ = 3 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time E > remaining time Z.

➔ Αναστέλλεται η διεργασία Z και να πάει στην ουρά αναμονής με τις διεργασίες Δ, A και B.

➔ Αρχίζει η εκτέλεση της διεργασίας E αφού έχει τον μεγαλύτερο απομένοντα χρόνο εκτέλεσης.

6) $t = 14 \text{ msec}$

ισχύει ότι remaining time Δ = 3msec και remaining time remaining time E = A = remaining time B = remaining time Z = 2 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time Δ > remaining time E.

➔ Αναστέλλεται η διεργασία E και να πάει στην ουρά αναμονής με τις διεργασίες Z, A και B.

➔ Αρχίζει η εκτέλεση της διεργασίας Δ αφού έχει τον μεγαλύτερο απομένοντα χρόνο εκτέλεσης.

7) $t = 19 \text{ msec}$

ισχύει ότι remaining time Δ = 2msec και remaining time E = remaining time A = remaining time B = remaining time Z = 2 msec.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time Δ > remaining time E.

➔ Αναστέλλεται η διεργασία E και να πάει στην ουρά αναμονής με τις διεργασίες Z, A και B.

➔ Αρχίζει η εκτέλεση της διεργασίας Δ αφού έχει τον μεγαλύτερο απομένοντα χρόνο εκτέλεσης.

8) $t = 25 \text{ msec}$

ισχύει ότι remaining time Δ = 1msec και οι υπόλοιπες διεργασίες έχουν εκτελεστεί.

➔ Άρα ενεργοποιείται η λειτουργία LTRFP αφού remaining time Δ = 1msec

➔ Ολοκληρώνεται η εκτέλεση της διεργασίας Δ.

PID explanation:

1) $t = 2 \text{ msec}$:

Φτάνει στο σύστημα η διεργασία Γ με χρόνο εκτέλεσης 1 msec και την ίδια στιγμή ισχύει

remaining time A = remaining time B = 4msec

➔ Άρα ενεργοποιείται η PID priority αφού δεν ικανοποιείται η LRTFP για την διεργασία Γ και $PID_A = 3$ ενώ $PID_B = 1$

➔ Άρα η διεργασία A αναστέλλεται, πηγαίνει στην ουρά αναμονής και ξεκινά η εκτέλεση της B.

2) $t = 8 \text{ msec}$:

remaining time Z = remaining time E = 4msec

➔ Άρα ενεργοποιείται η PID priority και $PID_E = 3$ ενώ $PID_Z = 1$

➔ Άρα η διεργασία Z συνεχίζει να εκτελείται.

3) $t = 10 \text{ msec}$:

remaining time A = remaining time B = remaining time Δ = remaining time Z = remaining time E = 3 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_B = 1$, $PID_Δ = 5$, $PID_E = 4$ και $PID_Z = 1$

➔ Άρα η διεργασία E αναστέλλεται, πηγαίνει στην ουρά αναμονής και ξεκινά η εκτέλεση της B ²⁷.

4) $t = 11 \text{ msec}$:

remaining time A = remaining time Δ = remaining time Z = remaining time E = 3 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_Δ = 5$, $PID_E = 4$ και $PID_Z = 1$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η Z.

5) $t = 12 \text{ msec}$:

remaining time A = remaining time Δ = remaining time E = 3 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_Δ = 5$ και $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η A.

6) $t = 13 \text{ msec}$:

remaining time Δ = remaining time E = 3 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_Δ = 5$ ενώ $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η E.

7) $t = 15 \text{ msec}$:

remaining time A = remaining time B = remaining time Δ = remaining time Z = remaining time E = 2 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_B = 1$, $PID_Δ = 5$, $PID_E = 4$ και $PID_Z = 1$

➔ Άρα η διεργασία Δ αναστέλλεται, πηγαίνει στην ουρά αναμονής και συνεχίζει η εκτέλεση της B ⁶.

8) $t = 16 \text{ msec}$:

remaining time A = remaining time Δ = remaining time Z = remaining time E = 2 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_Δ = 5$, $PID_E = 4$ και $PID_Z = 1$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η Z.

9) $t = 17 \text{ msec}$:

remaining time A = remaining time Δ = remaining time E = 2 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_Δ = 5$ και $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η A.

10) $t = 18 \text{ msec}$:

remaining time Δ = remaining time E = 2 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_Δ = 5$ ενώ $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η E.

11) $t = 20 \text{ msec}$:

remaining time A = remaining time B = remaining time Γ = remaining time Δ = remaining time Z = remaining time E = 1 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_B = 1$, $PID_Γ = 2$, $PID_Δ = 5$, $PID_E = 4$ και $PID_Z = 1$

²⁷ Επίδρασή ουράς στην σειρά εναλλαγής των διεργασιών αφού $PID_Z = PID_B$ και η διεργασία B αφίχθηκε στο σύστημα πριν από την Z.

➔ Άρα η διεργασία Δ αναστέλλεται , πηγαίνει στην ουρά αναμονής και συνεχίζει η εκτέλεση της Β ⁶.

12) $t = 21 \text{ msec}$:

remaining time A = remaining time Γ = remaining time Δ = remaining time Z = remaining time E = 1 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_\Gamma = 2$, $PID_\Delta = 5$, $PID_E = 4$ και $PID_Z = 1$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η Z.

13) $t = 22 \text{ msec}$:

remaining time A = remaining time Γ = remaining time Δ = remaining time E = 1 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_\Gamma = 2$, $PID_\Delta = 5$ και $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η Γ.

14) $t = 23 \text{ msec}$:

remaining time A = remaining time Δ = remaining time E = 1 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_A = 3$, $PID_\Delta = 5$ και $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η A.

15) $t = 24 \text{ msec}$:

remaining time Δ = remaining time E = 1 msec

➔ Άρα ενεργοποιείται η PID priority και $PID_\Delta = 5$ ενώ $PID_E = 4$.

➔ Άρα η επόμενη προς εκτέλεση διεργασία είναι η E.

✓ Χρόνοι Διεκπεραίωσης:

$$X_{\Delta A} = 24 - 0 = 24 \text{ msec} \quad X_{\Delta \Delta} = 26 - 3 = 23 \text{ msec}$$

$$X_{\Delta B} = 21 - 0 = 21 \text{ msec} \quad X_{\Delta E} = 25 - 4 = 21 \text{ msec}$$

$$X_{\Delta \Gamma} = 23 - 2 = 21 \text{ msec} \quad X_{\Delta Z} = 22 - 5 = 17 \text{ msec}$$

✓ Χρόνοι αναμονής :

$$X_{A_A} = (3 - 2) + (12 - 4) + (17 - 13) + (23 - 18) = 1 + 8 + 4 + 5 = 18 \text{ msec}$$

$$X_{A_B} = (2 - 0) + (10 - 3) + (15 - 11) + (20 - 16) = 2 + 7 + 4 + 4 = 17 \text{ msec}$$

$$X_{A_\Gamma} = 22 - 2 = 20 \text{ msec}$$

$$X_{A_\Delta} = (14 - 3) + (19 - 15) + (25 - 20) = 11 + 4 + 5 = 20 \text{ msec}$$

$$X_{A_E} = 0 + (9 - 5) + (13 - 10) + (18 - 14) + (24 - 19) = 4 + 3 + 4 + 5 = 16 \text{ msec}$$

✓ Χρόνοι απόκρισης :

$$X_A = 0 \text{ msec}$$

$$X_B = 2 \text{ msec}$$

$$X_\Gamma = 20 \text{ msec}$$

$$X_\Delta = 11 \text{ msec}$$

$$X_E = 0 \text{ msec}$$

$$X_Z = 0 \text{ msec}$$

Άρα :

- ✓ Μέσος Χρόνος Αναμονής = $\frac{XA_A + XA_B + XA_\Gamma + XA_\Delta + XA_E + XA_Z}{6} = \frac{18 + 17 + 20 + 20 + 16 + 10}{6} \approx 16.833\text{msec}$
- ✓ Μέσος Χρόνος Ολοκλήρωσης = $\frac{X\Delta_A + X\Delta_B + X\Delta_\Gamma + X\Delta_\Delta + X\Delta_E + X\Delta_Z}{6} = \frac{24 + 21 + 21 + 23 + 21 + 17}{6} \approx 21.16\text{msec}$
- ✓ Μέσος Χρόνος Απόκρισης = $\frac{0 + 2 + 20 + 11 + 0 + 0}{6} = 5.5 \text{ msec}$
- ✓ Πλήθος αλλαγών διεργασίας που εκτελείται στον επεξεργαστή (context switch): 22